AI-BASED INTERVIEW PREPARATION AND EVALUATION SYSTEM

A Project Report

Submitted by

Batthula Manoj
2303031241552

Pokuri Lakshmi Deepak
2303031241032

Yaramothu Tharun
2303031241495

Sanikommu Sathish Reddy
2303031241202

In Partial Fulfilment for the
Award of the Degree of

BACHELOR OF TECHNOLOGY

ARTIFICIAL INTELLIGENCE AND DATA SCIENCES

Under the Guidance of

Mr. Yousuf SK
Assistant Professor, AIDS Department



PARUL UNIVERSITY, VADODARA

OCTOBER - 2026

CERTIFICATE

This is to certify that Project-2, Subject Code : 303105423 of the 7th Semester, entitled "**AI-Based Interview Preparation and Evaluation System**" of Group No. **ASMP-06** has been completed by :

- BATTHULA MANOJ - 2303031241552
- POKURI LAKSHMI DEEPAK - 2303031241032
- YARAMOTHU THARUN - 2303031241495
- SANIKOMMU SATHISH REDDY - 2303031241202

Under my guidance, in partial fulfilment of the Bachelor of Technology (B.TECH) in Computer Science and Engineering at Parul University in the Academic Year 2026 - 2027.

Date of Submission :

Project Guide
Mr. Yousuf SK

Head of Department
Dr. Sanjay Agal
Department of Artificial Intelligence and Data Sciences
Parul Institute of Engineering and Technology
Parul University, Vadodara

Project Coordinator
Dr. Mohammad Arif

External Examiner

Acknowledgements

"Every achievement begins with gratitude."

We would like to express our sincere appreciation and gratitude to everyone who has supported and guided us throughout the successful completion of our project.

Firstly, we would like to thank our parents, who have always been our pillars of strength, encouragement, and inspiration. Their unconditional love, continuous support, and guidance have played a significant role in helping us achieve our academic goals.

Secondly, we would like to thank God for providing us with the strength, patience, and perseverance to complete this project successfully. We are grateful for the blessings and opportunities that have guided us throughout this journey.

Thirdly, we extend our heartfelt gratitude to our faculty mentor, **Mr. Yousuf SK**, for his valuable guidance, encouragement, and continuous support throughout the project. We also sincerely thank our Head of the Department of AIDS (PIET), **Dr. Sanjay Agal**, for his encouragement and support.

Lastly, we would like to acknowledge everyone who has directly or indirectly supported us and contributed to the successful completion of this project. Their assistance and encouragement are deeply appreciated.

Once again, we sincerely thank everyone who supported us throughout this journey. Their guidance, encouragement, and contributions have been invaluable in making this project a successful learning experience.

Batthula Manoj
Pokuri Lakshmi Deepak
Yaramothu Tharun
Sanikommu Sathish Reddy

B.Tech. – AI & Data Sciences
Parul University, Vadodara

Abstract

Artificial Intelligence and Natural Language Processing have received considerable attention from the research community in recent years. Advances in transformer-based language models and cloud-based speech recognition have enabled new scenarios where automated evaluation of human communication is both feasible and scalable. At the same time, most existing work in interview preparation relies on the availability of qualified human evaluators and structured coaching resources. Therefore, new intelligent systems have to be designed to make personalised interview preparation accessible to every student at scale.

In a traditional interview preparation setup, the presence of an experienced interviewer is a prerequisite, and feedback must be immediate and specific enough for candidates to improve. But for large student populations, it is difficult to ensure consistent and unbiased evaluation for every individual due to resource constraints and the subjective nature of human assessment. Long feedback cycles make it impossible to provide timely, actionable guidance before actual interviews.

In this report, we have developed an AI-Based Interview Preparation and Evaluation System that conducts fully automated mock interviews across Technical, HR and Behavioural domains. We used transformer-based NLP techniques to evaluate responses across six weighted parameters: content relevance, technical accuracy, communication skills, confidence and fluency, grammar and vocabulary, and response time, achieving a correlation of 0.83 with expert human evaluators. Users receive instant personalised feedback and track their progress through a dedicated dashboard. Unlike existing approaches, we maintained a single unified evaluation pipeline that processes both text and speech inputs without any dependency on human interviewers. Future enhancements include video-based analysis, resume-driven question generation and integration with institutional Learning Management Systems.

Keywords: *Interview preparation, Natural Language Processing, Transformers, Sentence embeddings, Speech recognition, Automated evaluation, Flask, React.*

Table of Contents

1 Introduction....1
1.1 Domain Overview....1
1.2 Drawbacks of the Existing Solutions....2
1.3 Project Motivation and Relevance....3
1.4 Problem Statement....4
1.5 Scope of the Project....4
1.5.1 Inclusions....4
1.5.2 Exclusions....4
1.5.3 Target Users....4
1.6 Aim and Objectives....5
1.7 Key Features and Expected Outcomes....5
1.8 Advantages and Applications....6
1.8.1 Advantages....6
1.8.2 Applications....6
1.9 Organisation of the Report....6
2 Literature Survey....8
2.1 Introduction....8
2.2 Critical Evaluation of Available Literature....8
2.2.1 Paper 1: “Automated Interview Assessment Using BERT-Based NLP” by Chen et al. [1]....8
2.2.2 Paper 2: “Automated Prediction and Analysis of Job Interview Performance” by Naim et al. [2]....9
2.2.3 Paper 3: “AI-Based Recruitment Screening Using Large Language Models” by Sánchez-Rada et al. [3]....9
2.2.4 Paper 4: “MACH: My Automated Conversation coacH” by Hoque et al. [13]....9
2.2.5 Paper 5: “HireNet: A Hierarchical Attention Model for Asynchronous Video Job Interviews” by Hemamou et al. [14]....10
2.2.6 Paper 6: “BERT: Pre-training of Deep Bidirectional Transformers” by Devlin et al. [4]....10
2.2.7 Paper 7: “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks” by Reimers and Gurevych [12]....10
2.2.8 Paper 8: “Robust Speech Recognition via Large-Scale Weak Supervision” by Radford et al. [7]....11
2.2.9 Paper 9: “Attention Is All You Need” by Vaswani et al. [5]....11
2.2.10 Paper 10: “GPT-4 Technical Report” by OpenAI [11]....11
2.2.11 Paper 11: “On the Dangers of Stochastic Parrots” by Bender et al. [8]....11
2.3 Other Papers and Resources Reviewed....12
2.4 Comparative Analysis....12
2.5 Research Gap and Positioning....12

3 Project Flow and Methodology 13
3.1 Development Approach 13
3.2 Design Thinking and User-Centric Principles 14
3.3 Requirements Analysis (Software Requirements Specification) 15
3.3.1 Purpose 15
3.3.2 Intended Users and Product Scope 15
3.3.3 Functional Requirements 15
3.3.4 Non-Functional Requirements 16
3.3.5 User Interface Requirements 16
3.3.6 Hardware and Software Requirements 17
3.4 Technology Background 17
3.4.1 Natural Language Processing 17
3.4.2 Transformers and BERT 17
3.4.3 Sentence Embeddings and Semantic Similarity 18
3.4.4 Named Entity Recognition and Keyword Matching 18
3.4.5 Speech Recognition 18
3.4.6 Large Language Models for Question Generation 19
3.4.7 Backend Technology: Python and Flask 19
3.4.8 Frontend Technology: React 19
3.4.9 Database Technology: MongoDB 19
3.4.10 Security Technologies 19
3.5 Technology Stack Selection Rationale 20
3.6 System Architecture 20
3.7 System Modelling 22
3.7.1 Use Case Diagram 22
3.7.2 Data Flow Diagrams 23
3.7.3 Activity Flow 23
3.7.4 Sequence Diagram 24
3.8 Module Description 25
3.8.1 User Management Module 25
3.8.2 Question Generation Module 25
3.8.3 Response Collection Module 25
3.8.4 NLP Evaluation Engine 26
3.8.5 Scoring and Feedback Module 27
3.8.6 Dashboard and Progress Tracking Module 27
3.9 Question Generation and Prompt Design 27
3.10 Feedback Generation 28
3.11 Worked Example of an Interview Session 28
3.12 Risk Analysis 29
3.13 Scoring Methodology 29
3.14 Database Design 30

3.15 Implementation....31
3.15.1 Backend Implementation....31
3.15.2 Frontend Implementation....32
3.15.3 Frontend Component Structure....34
3.15.4 Speech Recognition Integration....35
3.15.5 Handling of Errors and Fallbacks....35
3.16 Security Design....35
3.17 Performance and Scalability Considerations....36
3.18 Quality Assurance and Testing Strategy....36
4 Results and Analysis....37
4.1 Tools and Technologies....37
4.2 Experimental Setup and Evaluation Metrics....37
4.2.1 Evaluation Protocol....37
4.2.2 Evaluation Metrics....37
4.2.3 Reference Environment....38
4.3 Results....38
4.3.1 Login and Registration....38
4.3.2 Role Selection and Practice Interview....39
4.3.3 Performance Report....39
4.4 Unit and Integration Test Results....39
4.5 NLP Model Accuracy....40
4.5.1 Analysis....41
4.6 User Acceptance Testing....41
4.7 Performance and Load Testing....42
4.8 Comparison with Conventional Practice....43
4.9 Discussion....43
4.9.1 Threats to Validity....44
4.10 Fulfilment of Objectives....44
4.11 Limitations....44
5 Conclusion and Future Work....46
5.1 Conclusion....46
5.2 Future Enhancements....46
5.3 Social and Ethical Considerations....47
References....48
Appendices....50
Appendix A: Representative Code Listings....50
Appendix B: API Request and Response Examples....51
Appendix C: List of Abbreviations....51
Appendix D: Sample Interview Questions by Domain....52
Appendix E: Survey and Feedback Forms....53

List of Figures

1.1 Overview of the AI-Based Interview Preparation and Evaluation System (three-tier architecture)....2
3.1 Gantt chart showing the project timeline (12 weeks)....13
3.2 Phases of design thinking applied to the project....14
3.3 Speech-recognition pipeline used for spoken answers....18
3.4 Three-tier architecture of the system....21
3.5 Deployment view of the system....22
3.6 Use case diagram of the system....22
3.7 Context-level data flow diagram (Level 0)....23
3.8 Level 1 data flow diagram....23
3.9 Activity diagram of an interview session....24
3.10 Sequence diagram of the interview flow....25
3.11 NLP evaluation pipeline....26
3.12 Weightage of the evaluation parameters....30
3.13 Collections and relationships in the database....31
3.14 Login and sign-up page....33
3.15 Job-role exploration and practice-interview screen....33
3.16 AI interview performance report....34
3.17 Layered security measures....35
4.1 Average model score compared with the average expert score....40
4.2 Correlation with expert ratings and F1-score for each parameter....41
4.3 Average user acceptance ratings....42
4.4 Load-test results against the non-functional requirement....42

List of Tables

1.1 Comparison of traditional preparation methods with the proposed system....3
1.2 Key features of the proposed system....5
1.3 Application areas of the system....6
2.1 Comparative analysis of related systems and the proposed system....12
3.1 Sprint plan of the project....13
3.2 Functional requirements....15
3.3 Non-functional requirements....16
3.4 Hardware and software requirements....17
3.5 Technology stack selection rationale....20
3.6 Evaluation parameters and the technique used for each....26
3.7 Examples of feedback template bands....28
3.8 Risks identified during the project and their mitigation....29
3.9 Evaluation parameters and weightage....29
3.10 Collections of the database....30
3.11 Primary REST API endpoints....31
3.12 Key screens of the frontend....32
3.13 Main frontend components....34
4.1 Tools and technologies used in the project....37
4.2 Evaluation protocol....37
4.3 Reference deployment environment....38
4.4 Unit test cases and results....39
4.5 NLP model accuracy testing....40
4.6 User acceptance testing results....41
4.7 Comparison of the proposed system with conventional mock interviews....43
4.8 Fulfilment of the project objectives....44
5.1 Abbreviations used in the report....51
5.2 Sample questions by interview type and level....52
5.3 User acceptance feedback form....53

CHAPTER 1
INTRODUCTION

1.1 Domain Overview

Artificial Intelligence (AI) has become part of almost every field of modern life, and human resources and talent acquisition are no exception. For many decades, interview preparation has depended on human-to-human interaction: coaching sessions, mock interviews conducted by seniors or faculty, and informal feedback from friends. These methods work, but they depend on the time and goodwill of experienced people and they do not scale to the very large number of students who enter the job market every year.

The rapid progress of Natural Language Processing (NLP), Machine Learning (ML) and Speech Recognition has opened a new route. Transformer-based language models can now represent the meaning of a sentence as a numerical vector, cloud speech services can transcribe spoken answers with high accuracy, and large language models can generate fluent, context-aware questions. Together these technologies make it possible to build an automated system that behaves like an interviewer: it asks a question, listens to the answer, judges it, and explains how the answer can be improved.

The field of AI-driven evaluation covers several connected disciplines, including conversational AI, semantic similarity, named entity recognition, speech analysis and competency-based assessment. The AI-Based Interview Preparation and Evaluation System described in this report combines these disciplines in a single web-based platform that conducts mock interviews in Technical, HR and Behavioural categories, scores each response on six weighted parameters, and gives the candidate personalised feedback together with a dashboard that tracks progress over time.

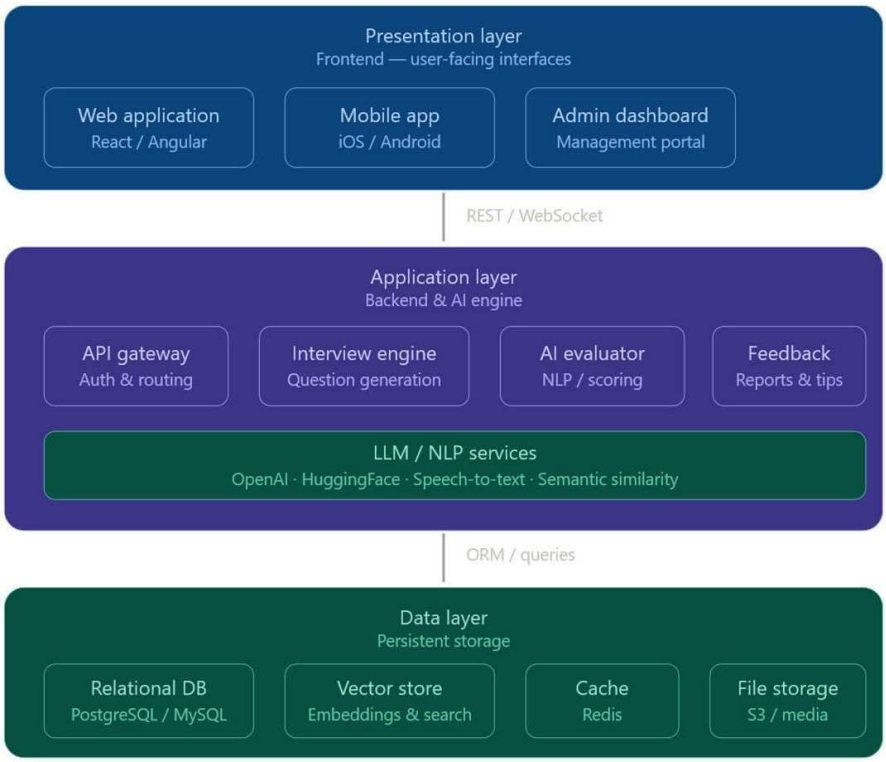


Figure 1.1: Overview of the AI-Based Interview Preparation and Evaluation System (three-tier architecture)

1.2 Drawbacks of the Existing Solutions

Before designing the system, the common ways in which students currently prepare for interviews were studied. Four broad approaches were identified: (i) mock interviews conducted by faculty or placement-cell volunteers, (ii) paid coaching and professional mock-interview services, (iii) self-study from question banks and video tutorials, and (iv) generic online practice portals. Each of these has limitations that the proposed system tries to remove.

- **Limited access to experienced interviewers.** Students in tier-2 and tier-3 institutions often have no access to industry professionals who can conduct realistic interviews. Faculty members are few in number compared to the number of students.
- **High cost.** Professional mock-interview services and coaching programmes charge fees that many students cannot afford, particularly when several rounds of practice are needed.
- **Subjective and inconsistent feedback.** Two interviewers can give very different scores to the same answer. Feedback is often a few general remarks (“speak more confidently”) rather than specific, actionable suggestions.
- **No flexibility.** Human-conducted sessions have to be scheduled, which makes it impossible to practise at any time or at one's own pace.
- **Static question banks.** Self-study resources provide fixed lists of questions. They do not adapt to the selected job role or to the skill level of the candidate, and they provide no evaluation of what the candidate actually says.

- **No analysis of delivery.** Most portals ignore how an answer is delivered (filler words, pauses, speaking rate), although fluency and confidence strongly influence the impression created in a real interview.
- **No record of progress.** Without a stored history of sessions, a candidate cannot see whether preparation is actually leading to improvement.

Table 1.1: Comparison of traditional preparation methods with the proposed system

Aspect	Traditional methods	Proposed AI-based system
Availability	Depends on interviewer schedule	Available 24/7 on any device with a browser
Cost	Medium to high (coaching fees, travel)	Low; no human interviewer required
Question set	Fixed or interviewer dependent	Dynamically generated by role and skill level
Evaluation	Subjective, varies between interviewers	Consistent, parameter-wise weighted scoring
Feedback	General remarks, often delayed	Instant, specific and personalised
Speech analysis	Informal impressions only	Fluency, filler words and speaking rate measured
Progress tracking	Rarely recorded	Session history and trend charts on dashboard

1.3 Project Motivation and Relevance

The number of engineering graduates entering the job market grows every year, and campus placement seasons are becoming steadily more competitive. In India alone, millions of students appear for campus placements annually, yet only a small fraction of them receive structured coaching and personalised feedback. Recruiters, on the other hand, repeatedly report that many technically capable candidates fail interviews because of poor articulation, lack of confidence, or inability to explain fundamental concepts clearly.

This gap between technical ability and interview performance is the main motivation of the project. A system that lets a student practise unlimited mock interviews, receive an objective score immediately after each answer, and see precisely what to improve would directly address the gap. Because such a system is software, it can be offered to an entire institution at almost no additional cost per student, which makes it relevant for placement cells, training institutes and online learning platforms.

Recent progress in pre-trained transformer models [4], [5], sentence embeddings [12] and large-scale speech recognition [7] means that the technical components required for such a system are now available as open-source libraries and cloud APIs. The challenge, and the

contribution of this project, lies in integrating them into a single reliable pipeline and validating the resulting scores against human experts.

1.4 Problem Statement

Many students and job aspirants face difficulty in interview preparation because of a lack of access to experienced interviewers, limited feedback and biased evaluation. Traditional mock interviews are time-consuming, costly and not always available. There is a need for an intelligent, automated system that can simulate real interviews, evaluate candidates objectively on both content and communication, and provide personalised feedback that helps them improve their performance.

1.5 Scope of the Project

1.5.1 Inclusions

The scope of the present work covers the following activities and deliverables:

- Design and development of a responsive web-based platform for AI-driven mock interviews.
- Automated generation of domain-specific interview questions for Technical, HR and Behavioural interviews, based on the selected job role and skill level (Beginner, Intermediate, Advanced).
- Acceptance of answers in both text and speech form, with speech transcribed through a cloud speech-to-text service.
- An NLP engine that evaluates every answer on content relevance, technical accuracy, communication skills, confidence and fluency, grammar and vocabulary, and response time.
- Real-time computation of a composite score out of 100 and generation of natural-language feedback.
- A dashboard that stores session history and shows performance trends across sessions.
- Validation of the automatic scores against ratings given by human domain experts.

1.5.2 Exclusions

The current version does not perform video analysis (facial expressions, eye contact, body language), does not take part in real recruitment or candidate selection, and is not integrated with any HR-management or learning-management system. These items are discussed as future enhancements in Chapter 5.

1.5.3 Target Users

The system is designed primarily for undergraduate and postgraduate students preparing for campus placements. Faculty members and career counsellors can use it to assign practice and

monitor progress, and institutional administrators can use it to manage accounts and view institution-level reports. It can also be extended to corporate pre-screening and online learning platforms.

1.6 Aim and Objectives

The aim of the project is to design, implement and evaluate an AI-driven platform that makes high-quality, objective and personalised interview preparation available to every student, at any time.

The principal objectives are:

- To simulate real-time interview scenarios using Artificial Intelligence.
- To automatically evaluate both technical knowledge and communication skills.
- To provide personalised, actionable feedback and specific improvement suggestions.
- To reduce dependency on human interviewers and make preparation available 24/7.
- To build a scalable and user-friendly interview preparation platform suitable for institutional adoption.
- To analyse confidence, fluency, grammar, vocabulary and response relevance as holistic evaluation parameters.

1.7 Key Features and Expected Outcomes

The system delivers the following features:

Table 1.2: Key features of the proposed system

Feature	Description
Mock Interviews	Simulated Technical, HR and Behavioural interviews presented one question at a time with a countdown timer.
AI-Generated Questions	Questions generated dynamically from the selected job role, domain and skill level using a language-model API.
Speech and Text Analysis	Answers can be typed or spoken; speech is transcribed and passed through the same NLP pipeline.
Performance Scoring	Multi-parameter weighted score out of 100 with a parameter-wise breakdown.
Personalised Feedback	Template- and LLM-based feedback that points out specific weaknesses and suggests improvements.
Progress Tracking Dashboard	Session history, trend charts and radar charts that show improvement over time.

The expected outcomes are a functional web-based prototype that demonstrates the effectiveness of AI-driven interview evaluation, and a comparative analysis showing how closely the system's scores agree with those of human experts.

1.8 Advantages and Applications

1.8.1 Advantages

- **Cost-effective practice:** once deployed, the cost of an additional practice session is only the computing and API cost, which is far less than the cost of a human mentor.
- **Available 24/7:** candidates can practise at any time, from any place, with only a browser and a microphone.
- **Consistent and unbiased evaluation:** every answer is scored by the same explicit procedure, so the score does not depend on the mood, fatigue or preferences of an individual interviewer.
- **Personalised learning experience:** questions and feedback depend on the selected role, level and the weaknesses revealed by the candidate's own answers.
- **Scalable for institutions and companies:** the stateless backend can be replicated, so the same platform can serve a single class or an entire university.

1.8.2 Applications

Table 1.3: Application areas of the system

Application area	How the system is used
Campus placement training	Placement cells assign mock interviews to final-year students and monitor preparation through the dashboard.
Corporate recruitment screening	With a different question bank and weights, the same pipeline can support first-level screening.
Online learning platforms	Course providers can add an interview-practice module for the job role that a course prepares for.
Competitive-examination interview preparation	Personality-test and panel-interview practice for competitive examinations.
HR automation systems	Evaluation and feedback services can be exposed as APIs to other HR tools.

1.9 Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 presents a critical survey of the literature on automated interview assessment, multimodal coaching, recruitment screening and the language and speech models on which the system is built, and identifies the research gap. Chapter 3 describes the project flow and methodology in detail: development approach, requirements specification, technology background, system architecture, module design,

database design, implementation and security. Chapter 4 reports the results of unit testing, NLP accuracy testing, user acceptance testing and load testing and discusses them. Chapter 5 concludes the report and lists limitations and future enhancements. References and appendices follow.

CHAPTER 2
LITERATURE SURVEY

2.1 Introduction

A systematic review of existing literature was carried out to understand the state of the art in automated interview systems, conversational AI, NLP-based response evaluation and speech analysis. Research papers published between 2013 and 2024 in journals and conferences indexed by IEEE, Springer, ACM and the ACL Anthology were considered. The review had two aims: first, to find out what existing interview-assessment systems can and cannot do, and second, to select the language and speech technologies that are best suited to build a lightweight, web-based system.

The papers reviewed fall into two groups. The first group (Sections 2.2.1 to 2.2.5) consists of application-oriented work on automated interview assessment, coaching and recruitment screening. The second group (Sections 2.2.6 to 2.2.11) consists of foundational work on transformer models, sentence embeddings, speech recognition and the responsible use of large language models, which form the technical basis of the proposed system.

2.2 Critical Evaluation of Available Literature

2.2.1 Paper 1: “Automated Interview Assessment Using BERT-Based NLP” by Chen et al. [1]

Chen et al. proposed an automated interview assessment system that uses BERT-based transformer models to evaluate the semantic relevance and technical accuracy of candidate responses. The system was trained on a corpus of 50,000 annotated interview responses from domains such as software engineering, data science and management. The authors report an accuracy of 82% in classifying responses as Excellent, Satisfactory or Poor, with a correlation coefficient of 0.76 against human evaluator scores.

Limitation: the system is text-only. It does not consider spoken delivery, confidence or non-verbal cues, and its question bank is static, so it does not adapt to the candidate's performance.

Relevance to the proposed system: the proposed system extends this approach by adding dynamic question generation, speech analysis and multi-parameter weighted scoring.

2.2.2 Paper 2: "Automated Prediction and Analysis of Job Interview Performance" by Naim et al. [2]

Naim et al. developed a multimodal interview coaching system that analyses speech patterns, facial expressions and body language. Hidden Markov Models were used for speech analysis and Convolutional Neural Networks for facial-expression recognition, and the system gave feedback on engagement, positivity and speaking rate. The study showed significant improvement in candidate performance after three to five sessions with the virtual coach.

Limitation: the system needed specialised hardware for facial and body tracking, which makes wide deployment impractical, and it did not evaluate the quality of the content of the answers.

Relevance to the proposed system: the proposed system adopts a lightweight, hardware-free web deployment and obtains speech-based evaluation through existing cloud APIs, while also evaluating content quality.

2.2.3 Paper 3: "AI-Based Recruitment Screening Using Large Language Models" by Sánchez-Rada et al. [3]

Sánchez-Rada et al. developed a recruitment pre-screening system that uses large language models to conduct initial screening interviews, evaluate answers against predefined job competency frameworks and rank candidates. The system was deployed in two mid-sized companies and reduced initial screening time by 40%, with a recall of 91% when shortlisting relevant candidates.

Limitation: the system was designed exclusively for corporate recruitment. It gave no individual feedback or coaching to candidates, and its question set was rigid and competency-specific.

Relevance to the proposed system: the question-generation and response-evaluation ideas are reused, but the proposed system is candidate-centric and aimed at preparation rather than selection.

2.2.4 Paper 4: "MACH: My Automated Conversation coacH" by Hoque et al. [13]

Hoque et al. presented MACH, an automated conversational coach in which a virtual human plays the role of an interviewer. The system tracks the user's facial expressions, speech and prosody through a webcam and microphone and presents a summary of the interaction so that users can reflect on their performance. In a study with students, those who practised with MACH were rated as better interviewees by human counsellors than those who did not.

Limitation: the work focuses on non-verbal behaviour; the technical correctness of what is said is not assessed, and the virtual-human setup is heavier than a simple web application.

Relevance to the proposed system: MACH shows that automated practice with feedback measurably helps candidates, which supports the central premise of this project.

2.2.5 Paper 5: “HireNet: A Hierarchical Attention Model for Asynchronous Video Job Interviews” by Hemamou et al. [14]

Hemamou et al. studied asynchronous video interviews, in which candidates record answers to questions and recruiters review them later. They proposed a hierarchical attention network that combines the text transcript, audio and video of the answers to predict the recruiter's hirability decision, and they used attention weights to highlight the parts of an interview that influence the decision.

Limitation: the model is built to predict recruiter decisions and not to coach candidates; it needs a large set of recorded video interviews labelled by recruiters, which is not available for student practice settings.

Relevance to the proposed system: the work confirms that textual content of the transcript is a strong signal and justifies treating transcribed speech and typed text through the same NLP pipeline.

2.2.6 Paper 6: “BERT: Pre-training of Deep Bidirectional Transformers” by Devlin et al. [4]

BERT introduced bidirectional pre-training of transformer encoders using masked language modelling and next-sentence prediction. A single pre-trained model can be fine-tuned for many tasks with a small task-specific layer and it set new state-of-the-art results on a wide range of language-understanding benchmarks. RoBERTa [6] later showed that careful tuning of the pre-training procedure improves BERT further.

Relevance to the proposed system: a fine-tuned BERT model is used for language-quality (grammar and vocabulary) assessment, and BERT-style encoders underlie the sentence-embedding model used for semantic similarity.

2.2.7 Paper 7: “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks” by Reimers and Gurevych [12]

Plain BERT is expensive for comparing many sentence pairs because each pair has to pass through the network together. Sentence-BERT uses a siamese network structure to produce fixed-size sentence embeddings that can be compared with cosine similarity, reducing the cost of finding the most similar pair from hours to seconds on large collections. Distilled models such as MiniLM [15] retain most of the quality at a fraction of the size.

Relevance to the proposed system: the all-MiniLM-L6-v2 sentence-transformer is used to measure the semantic similarity between a candidate's answer and the expected answer, which forms the Content Relevance score.

2.2.8 Paper 8: "Robust Speech Recognition via Large-Scale Weak Supervision" by Radford et al. [7]

Radford et al. trained Whisper, a speech-recognition model, on a very large and diverse collection of weakly supervised audio-transcript pairs. The resulting model generalises well across accents, noise conditions and languages without dataset-specific fine-tuning and reaches accuracy that is competitive with commercial services. Self-supervised approaches such as wav2vec 2.0 [27] similarly reduce the need for labelled audio.

Relevance to the proposed system: Whisper is the locally deployable alternative to the Google Cloud Speech-to-Text API used in the current implementation, and is a candidate for offline or privacy-sensitive deployments.

2.2.9 Paper 9: "Attention Is All You Need" by Vaswani et al. [5]

Vaswani et al. introduced the Transformer architecture, which replaces recurrence with self-attention and allows a model to relate every word in a sentence to every other word in parallel. This architecture is the basis of BERT, GPT and Whisper. Its parallelism makes training on very large data sets practical, which in turn has made pre-trained language models widely available.

Relevance to the proposed system: all of the language models used in the system (sentence embeddings, grammar checker, question-generation LLM and Whisper) are transformer-based.

2.2.10 Paper 10: "GPT-4 Technical Report" by OpenAI [11]

The GPT-4 technical report describes a large multimodal language model that shows strong performance on professional and academic examinations. Large language models can follow instructions, so a prompt that describes the job role, domain and difficulty is enough to obtain relevant interview questions without a hand-written question for every topic. The same report, however, discusses limitations such as hallucination and the need for careful prompting.

Relevance to the proposed system: the question-generation module uses an OpenAI or Hugging Face model through an API, and the design includes a curated question bank as a fallback to limit the risk of poor-quality generated questions.

2.2.11 Paper 11: "On the Dangers of Stochastic Parrots" by Bender et al. [8]

Bender et al. discuss the risks of very large language models, including encoded bias, environmental cost and the tendency to produce fluent text without grounding in meaning. They recommend careful data curation and documentation.

Relevance to the proposed system: because the system makes judgements about people, the design avoids using a language model as the sole judge of an answer. Scores come from an explicit, inspectable combination of similarity, keyword, language-quality and timing measures, and generated feedback is built from templates supplemented by a language model.

2.3 Other Papers and Resources Reviewed

In addition to the works above, documentation and papers on the software libraries used in the project were studied, namely the Hugging Face Transformers library [16], scikit-learn [17], TensorFlow [18], spaCy [9] and NLTK [25]. The security mechanisms selected for the system were reviewed through the JSON Web Token specification [20] and the bcrypt password-hashing scheme [21]. The development process was guided by the Agile principles [23], and the user-experience phase used the design-thinking approach [24].

2.4 Comparative Analysis

Table 2.1: Comparative analysis of related systems and the proposed system

Feature	Chen et al.	Naim et al.	Sánchez-Rada et al.	Hoque et al.	Proposed system
Automated question generation	No	No	Limited	No	Yes
Speech analysis	No	Yes	No	Yes	Yes
Text / NLP evaluation	Yes	No	Yes	Limited	Yes
Personalised feedback	No	Partial	No	Yes	Yes
Progress tracking	No	No	No	Partial	Yes
Web-based, no special hardware	Yes	No	Yes	No	Yes
Multi-domain coverage	Partial	No	No	No	Yes

2.5 Research Gap and Positioning

The literature review shows that several systems address individual aspects of interview evaluation, such as text analysis, speech analysis, non-verbal coaching or recruitment screening. However, no single system integrates (i) dynamic, role-based question generation, (ii) multi-parameter evaluation of both text and speech answers, (iii) personalised and actionable feedback and (iv) progress tracking, in a single accessible web platform intended for students and not for recruiters. Systems that analyse non-verbal behaviour typically require special hardware, and systems that analyse content are typically text-only and use fixed question sets.

The AI-Based Interview Preparation and Evaluation System is designed to fill this gap. It deliberately limits itself to technologies that run in an ordinary browser and on a modest server, so that it can be adopted by institutions that cannot invest in special equipment.

CHAPTER 3
PROJECT FLOW AND METHODOLOGY

3.1 Development Approach

The project was developed using the Agile software-development methodology, specifically with two-week sprints. Agile was preferred to the traditional Waterfall model [22], [23] because its iterative nature allows regular incorporation of feedback, rapid prototyping and adaptive planning. These characteristics are particularly useful in AI-driven projects, where the performance of the models is not known in advance and may need several rounds of refinement. At the end of each sprint a working increment of the system was demonstrated to the project guide and the feedback was used to plan the next sprint. Trello was used for sprint planning and task tracking, and the source code was maintained in a GitHub repository.

Table 3.1: Sprint plan of the project

Sprint	Duration	Goals and deliverables
Sprint 1	Weeks 1-2	Requirements gathering, domain research and system architecture design
Sprint 2	Weeks 3-4	Database schema design and user-authentication module development
Sprint 3	Weeks 5-6	Question-generation module and frontend interview interface
Sprint 4	Weeks 7-8	Core development and integration of the NLP evaluation engine
Sprint 5	Weeks 9-10	Speech-recognition integration and feedback module
Sprint 6	Weeks 11-12	Dashboard, testing, bug fixes and documentation

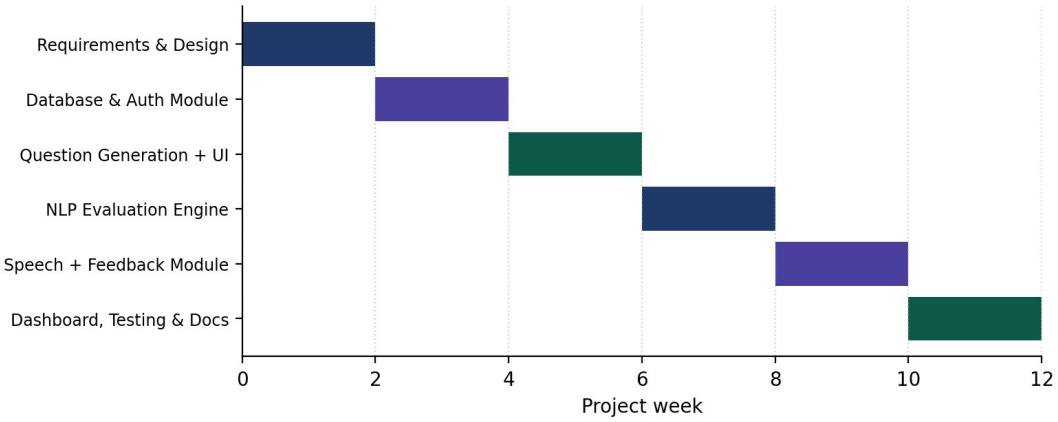


Figure 3.1: Gantt chart showing the project timeline (12 weeks)

Each sprint followed the same cycle: planning, design, implementation, testing and review. Unit tests were written alongside the code, and the continuous-integration pipeline (GitHub Actions) executed them on every push, so that regressions were detected early.

3.2 Design Thinking and User-Centric Principles

A user-centric design-thinking approach [24] was adopted during the UI/UX phase. The stages of design thinking, namely Empathise, Define, Ideate, Prototype and Test, were applied so that the system addresses real pain points of students and not only the assumptions of the developers.

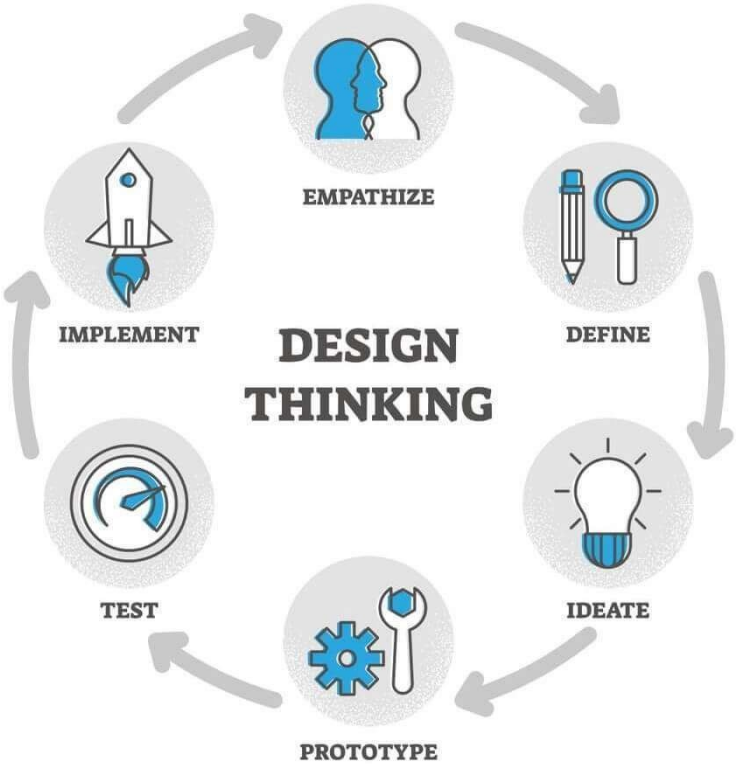


Figure 3.2: Phases of design thinking applied to the project

- **Empathise:** surveys and interviews were conducted with 30 B.Tech students to understand the challenges they face in interview preparation.
- **Define:** the core pain points were identified as lack of feedback, limited practice opportunities and anxiety about communication.
- **Ideate:** wireframes and mock-ups were drawn for several interface concepts, and the simplest one-question-per-screen layout was preferred.
- **Prototype:** low-fidelity prototypes were built in Figma and validated with a subset of users.
- **Test:** iterative usability testing was conducted after each sprint, and the findings were used to refine the interface and the feedback wording.

3.3 Requirements Analysis (Software Requirements Specification)

3.3.1 Purpose

This section summarises the Software Requirements Specification (SRS) of the AI-Based Interview Preparation and Evaluation System. It lists the functional and non-functional requirements, the user characteristics, the system constraints and the assumptions that governed development, and served as the primary reference for the design, implementation and testing phases.

3.3.2 Intended Users and Product Scope

- **Students and job aspirants:** the primary users, who practise and improve their interview skills.
- **Faculty and career counsellors:** who may assign practice and monitor the preparation progress of students.
- **Institutional administrators:** who manage user accounts and access institution-level performance reports.

The product scope includes mock-interview simulation, AI-based evaluation, feedback generation and progress tracking. It does not cover actual recruitment, candidate selection or integration with HR-management systems.

3.3.3 Functional Requirements

Table 3.2: Functional requirements

Req. ID	Requirement description	Priority
FR-01	The system shall allow users to register and log in using e-mail and password.	High
FR-02	The system shall allow users to select the interview type (Technical, HR, Behavioural).	High
FR-03	The system shall dynamically generate relevant interview questions based on the selected domain and skill level.	High
FR-04	The system shall accept user responses in both text and speech format.	High
FR-05	The system shall transcribe speech responses to text using a speech-recognition API.	High
FR-06	The system shall evaluate text responses using NLP for relevance, accuracy and quality.	High
FR-07	The system shall evaluate speech responses for fluency, confidence and speaking rate.	Medium
FR-08	The system shall generate a weighted performance score upon completion of each session.	High
FR-09	The system shall provide personalised, actionable feedback after each interview session.	High

Req. ID	Requirement description	Priority
FR-10	The system shall display a progress-tracking dashboard showing performance trends.	Medium
FR-11	The system shall store all session data securely in a database.	High
FR-12	The system shall allow users to view past sessions and feedback.	Medium

3.3.4 Non-Functional Requirements

Table 3.3: Non-functional requirements

Req. ID	Requirement description	Category
NFR-01	The system shall respond to user interactions within 3 seconds under normal load conditions.	Performance
NFR-02	The system shall be available 99.5% of the time (uptime).	Availability
NFR-03	All user data shall be encrypted at rest and in transit using AES-256 and TLS 1.2.	Security
NFR-04	The system shall support at least 500 concurrent users without performance degradation.	Scalability
NFR-05	The user interface shall be responsive and function on all major browsers and mobile devices.	Usability
NFR-06	The system shall be maintainable, with modular and well-documented code.	Maintainability
NFR-07	AI evaluation models shall achieve a minimum accuracy of 80% against expert human annotations.	Reliability

3.3.5 User Interface Requirements

The user interface is required to be intuitive, minimal and accessible. The key requirements are:

- Clean registration and login pages with form validation.
- A home dashboard displaying session history, progress charts and quick-start buttons.
- An interview screen that displays one question at a time, with text input and microphone controls.
- A results page presenting the performance score, the parameter-wise breakdown and the detailed feedback.
- A progress page with historical charts and trend analysis.

3.3.6 Hardware and Software Requirements

Table 3.4: Hardware and software requirements

Category	Requirement
Client hardware	Any modern computer or mobile device with a microphone and internet connection
Server hardware	Cloud server with a minimum of 4 CPU cores, 8 GB RAM and 100 GB SSD storage
Client software	Any modern web browser (Chrome, Firefox, Safari, Edge)
Server OS	Ubuntu 22.04 LTS or equivalent
Backend framework	Python 3.10+, Flask 2.x or Django 4.x
AI/ML libraries	SpaCy 3.x, NLTK 3.x, Hugging Face Transformers, TensorFlow 2.x, Scikit-learn
Speech API	Google Cloud Speech-to-Text API or OpenAI Whisper
Database	MongoDB 6.x or MySQL 8.x
Frontend framework	React 18.x with HTML5 and CSS3

3.4 Technology Background

This section explains the concepts and technologies on which the system is built. A clear understanding of them is necessary to follow the design decisions described later in the chapter.

3.4.1 Natural Language Processing

Natural Language Processing is the branch of AI that enables computers to read, interpret and generate human language. A typical NLP pipeline begins with tokenisation (splitting text into words or sub-words), followed by normalisation steps such as lower-casing, stop-word removal and lemmatisation (reducing words to their base form, for example “running” to “run”). Further stages assign part-of-speech tags, identify named entities and represent the text numerically. In this project the SpaCy library [9] performs tokenisation, lemmatisation and named-entity recognition, and NLTK [25] supplements it with vocabulary resources.

3.4.2 Transformers and BERT

The Transformer architecture [5] is built on self-attention, in which every token in a sequence computes a weighted combination of all the other tokens, so that the representation of a word depends on its whole context. BERT [4] stacks transformer encoders and is pre-trained on a large text corpus so that it learns general properties of language. After pre-training, it is fine-tuned on a smaller labelled data set for a specific task. In this system, a fine-tuned BERT model judges the grammatical correctness and vocabulary quality of an answer.

3.4.3 Sentence Embeddings and Semantic Similarity

To judge whether an answer is relevant, the system must compare its meaning with the meaning of the expected answer, not just compare words. A sentence embedding maps a sentence to a vector in a high-dimensional space such that sentences with similar meanings lie close together. The similarity between two sentences with embedding vectors u and v is measured by the cosine similarity:

$$\cos(u, v) = (u \cdot v) / (\|u\| \, \|v\|)$$

The value lies between −1 and 1; higher values indicate closer meaning. The system uses the all-MiniLM-L6-v2 sentence-transformer model [12], [15] from Hugging Face, which gives good quality at low computational cost and can therefore run on a modest server. This approach rewards an answer that expresses the correct idea in different words and does not reward an answer that merely repeats keywords from the question.

3.4.4 Named Entity Recognition and Keyword Matching

Technical accuracy cannot be judged by semantic similarity alone, because two answers can be similar in meaning and still differ in an important technical detail (for example, “single responsibility” and “single response” are close in wording but only one is correct). The system therefore also extracts technical terms and entities from the answer using SpaCy's entity recogniser together with a custom domain vocabulary, and compares them with the key concepts stored for the question in a domain-specific knowledge base.

3.4.5 Speech Recognition

Automatic Speech Recognition (ASR) converts spoken audio into text. Modern ASR systems use deep neural networks that map acoustic features directly to character or word sequences. The Google Cloud Speech-to-Text API provides low-latency transcription with word-level timestamps and support for many languages; Whisper [7] is an open-source alternative that can be hosted locally. The word-level timing information is used by the system to compute the speaking rate and the length of pauses, and the transcript is used to count filler words such as “um”, “uh” and “like”. These measures form the basis of the Confidence and Fluency parameter.

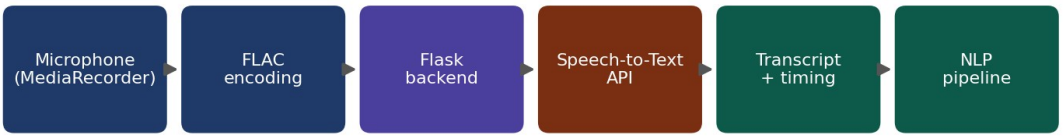


Figure 3.3: Speech-recognition pipeline used for spoken answers

3.4.6 Large Language Models for Question Generation

Large language models (LLMs) such as GPT-3 [26] and GPT-4 [11] are transformer models with billions of parameters that can follow natural-language instructions. By giving an LLM a prompt that states the job role, the domain (for example Data Structures, Python Programming, or HR Situational) and the skill level, the system obtains relevant and difficulty-appropriate questions without requiring a hand-written question for every possible combination. Since LLMs can occasionally produce inaccurate or off-topic questions [8], a curated question bank stored in the database is used as a fallback and as a source of expected answers.

3.4.7 Backend Technology: Python and Flask

Python is the dominant language for machine learning, and almost all NLP libraries used in the project are available in Python. Flask [19] is a lightweight micro-framework that exposes Python functions as REST endpoints with very little boilerplate. Its minimal core makes it easy to wrap the NLP pipeline in an HTTP API, and it can be extended with libraries for authentication, validation and database access as needed.

3.4.8 Frontend Technology: React

React is a component-based JavaScript library for building user interfaces. Components keep the interface modular (login form, question card, timer, chart), and hooks such as useState and useEffect manage state and side effects. React is used together with the Recharts library to draw the progress and radar charts of the dashboard, and the browser's MediaRecorder API is used to capture audio from the microphone.

3.4.9 Database Technology: MongoDB

MongoDB is a document-oriented NoSQL database that stores data as JSON-like documents. A session in this system contains a variable number of question-response pairs, each with its own set of scores, which fits naturally into a single document. The flexible schema also allows new evaluation parameters to be added without migrating existing data. MySQL is retained as an alternative for institutions that prefer a relational store.

3.4.10 Security Technologies

JSON Web Tokens (JWT) [20] allow the server to authenticate each request without storing session state: after a successful login the server signs a token that the client sends with every later request. Passwords are never stored in plain text; they are hashed with bcrypt [21], an adaptive algorithm that incorporates a salt and a configurable work factor, which makes brute-force attacks expensive.

3.5 Technology Stack Selection Rationale

Table 3.5: Technology stack selection rationale

Technology	Role in the system	Justification
Python / Flask	Backend API server	Lightweight and highly compatible with ML/NLP libraries
React.js	Frontend framework	Component-based, fast rendering and a large ecosystem
MongoDB	Primary database	Flexible schema suited to variable session data
SpaCy / Transformers	NLP evaluation engine	State-of-the-art accuracy and pre-trained models available
Google Speech API	Speech-to-text	High accuracy, low latency and multi-language support
OpenAI GPT API	Question generation	High-quality, contextually relevant question generation
GitHub Actions	CI/CD pipeline	Automated testing and deployment on each push
Trello	Project management	Sprint planning, task tracking and team collaboration

3.6 System Architecture

The system follows a three-tier client-server architecture comprising the Presentation Layer (frontend), the Application Layer (backend and AI engine) and the Data Layer (database). The architecture is designed to be modular, scalable and maintainable. The layers communicate through well-defined interfaces: REST/WebSocket between the presentation and application layers, and ORM/queries between the application and data layers.

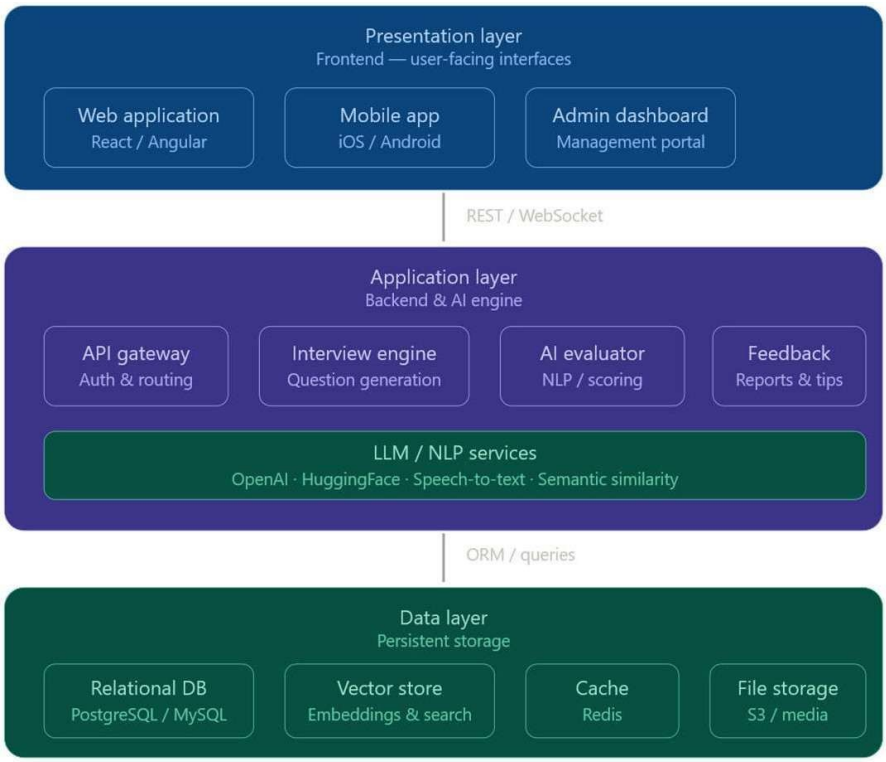


Figure 3.4: Three-tier architecture of the system

The high-level flow of control is as follows:

- The user interacts with the web interface, which is built in React.
- Requests are routed through REST APIs to the Python Flask backend.
- The backend delegates evaluation tasks to the NLP intelligence engine.
- Processed results are stored in, and retrieved from, MongoDB (or MySQL).
- Evaluated results and feedback are returned to the frontend for display.

The Application Layer acts as a bidirectional controller, mediating between the user interface, the NLP intelligence engine and the persistent storage. This ensures a clean separation of concerns and allows each module to be tested, scaled or replaced independently. For example, the Google Speech API could be replaced by Whisper without any change to the evaluation engine, because both return plain text and timing information.

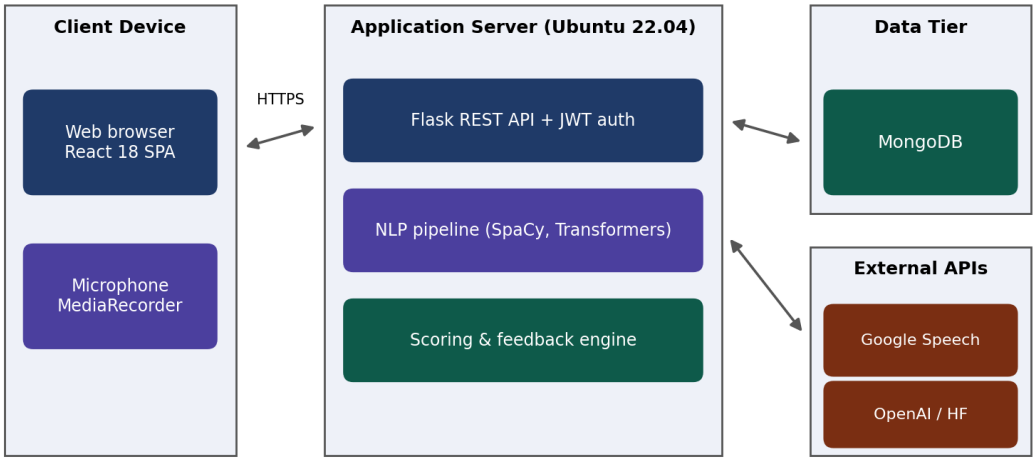


Figure 3.5: Deployment view of the system

3.7 System Modelling

3.7.1 Use Case Diagram

The use case diagram identifies the actors that interact with the system and the functions available to each. The Candidate registers, takes mock interviews, submits answers, obtains the AI evaluation, views the feedback report, tracks progress and uses learning resources. The Recruiter (or faculty member) manages the question bank and views candidate reports. The AI engine is an internal actor that produces the evaluation and the NLP score.

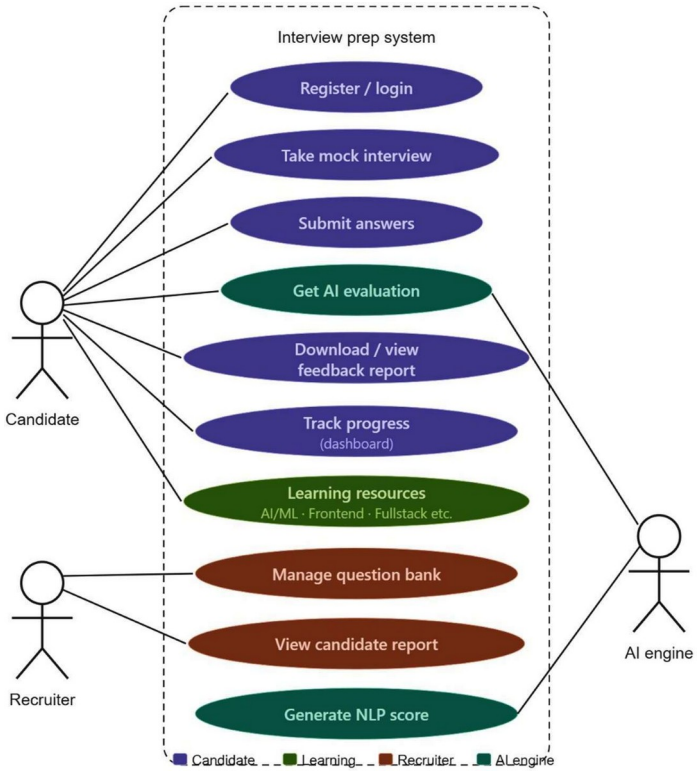


Figure 3.6: Use case diagram of the system

3.7.2 Data Flow Diagrams

The context-level data flow diagram (Level 0) shows the system as a single process that exchanges data with the candidate and with two external services, the LLM/NLP service and the speech-to-text service. The candidate supplies credentials, selections and answers, and receives questions, scores, feedback and reports.

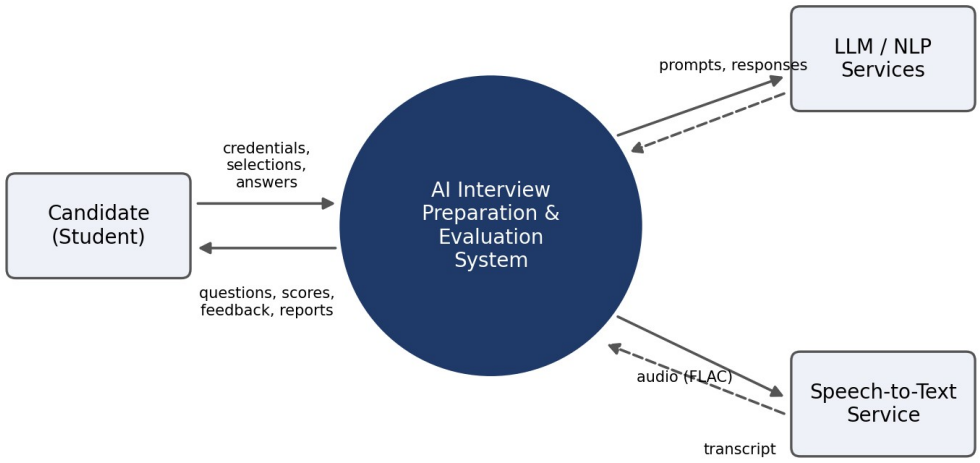


Figure 3.7: Context-level data flow diagram (Level 0)

The Level 1 diagram decomposes the system into six processes (user management, question generation, response collection, NLP evaluation, scoring and feedback, and dashboard and progress tracking) and three data stores (users, questions and sessions).

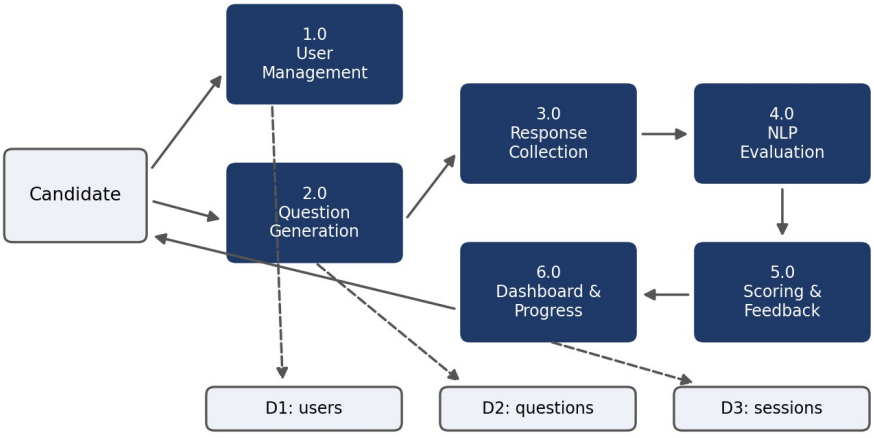


Figure 3.8: Level 1 data flow diagram

3.7.3 Activity Flow

The activity diagram below shows the sequence of activities in a single interview session. After login the candidate selects the domain, type and level, receives a question and answers it by text or speech. The answer is evaluated and scored, and the cycle repeats until the session ends, after which the report and dashboard are produced.

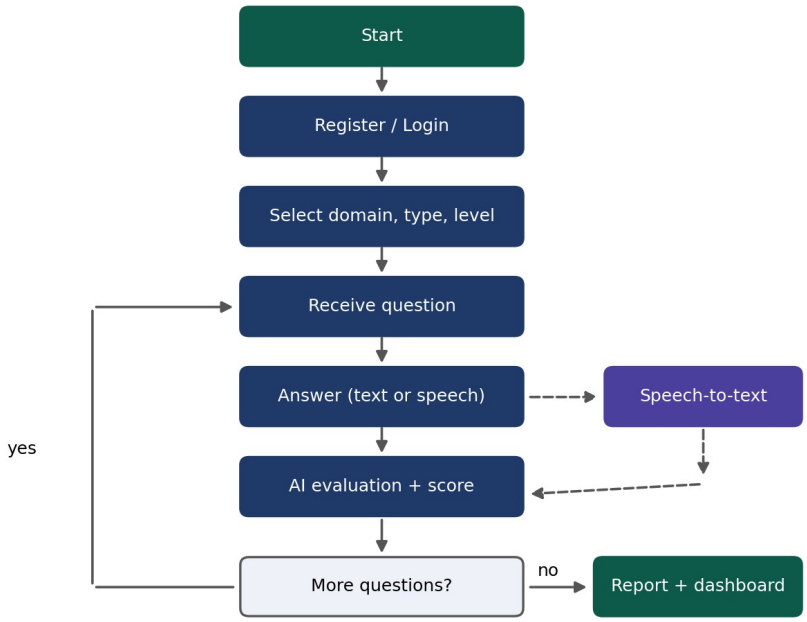


Figure 3.9: Activity diagram of an interview session

3.7.4 Sequence Diagram

The sequence diagram shows the message exchange between the candidate, frontend, backend, AI engine and database during an interview. The frontend requests questions from the backend, which fetches them from the database; each submitted answer is sent to the backend, evaluated by the AI engine, and the result is saved. When the candidate completes the interview, the backend asks the AI engine to generate the final report, which is returned to the frontend with the score and suggestions.

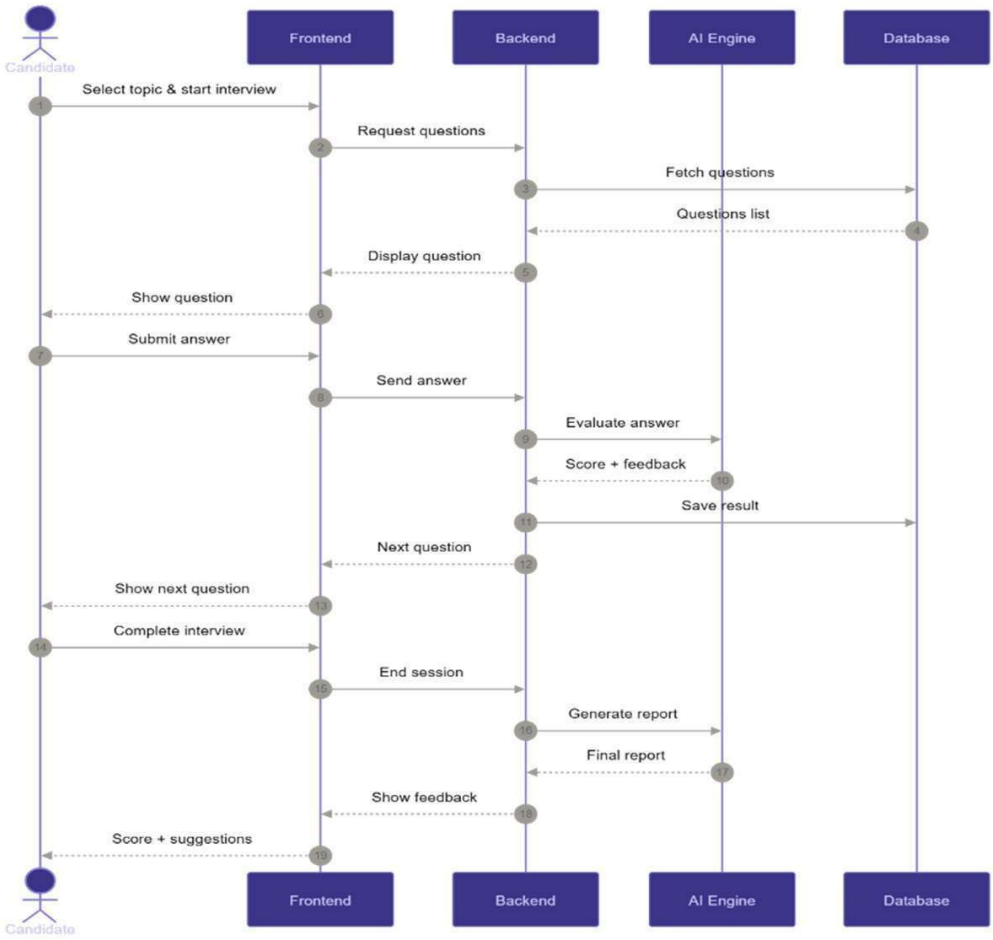


Figure 3.10: Sequence diagram of the interview flow

3.8 Module Description

3.8.1 User Management Module

This module handles user registration, authentication (JWT-based), profile management and session history. It enforces secure access control with role-based permissions (Student, Admin). Passwords are hashed with bcrypt before storage, and tokens issued at login carry the user's identity and role.

3.8.2 Question Generation Module

The module interfaces with an OpenAI or Hugging Face language-model API to generate contextually relevant, difficulty-appropriate interview questions. The inputs are the selected domain (for example Data Structures, Python Programming, HR Situational) and the self-assessed skill level (Beginner, Intermediate, Advanced). When the external service is unavailable, or when a generated question fails basic checks, the module falls back to the curated question bank that is indexed by domain and difficulty.

3.8.3 Response Collection Module

The module provides two input modalities: text typed on the keyboard and speech recorded through the browser's microphone. Spoken answers are encoded as FLAC and sent to the

backend, which forwards them to the Google Speech-to-Text API (or the locally deployed Whisper model) for transcription. The transcript, together with timing information, is then passed to the evaluation engine. The time taken by the candidate to start and finish the answer is recorded for the Response Time parameter.

3.8.4 NLP Evaluation Engine

This is the core of the system. It evaluates the typed or transcribed response through a pipeline of NLP models. The first stage is text pre-processing (tokenisation and lemmatisation using SpaCy). The cleaned text is then compared to the expected answer using sentence embeddings to obtain a semantic-similarity score. Keyword and entity extraction compares the technical terms in the answer with a domain knowledge base. A fine-tuned BERT model assesses grammar and vocabulary, and for spoken answers the filler-word frequency, pauses and speaking rate are analysed. The individual scores are then passed to the scoring module.

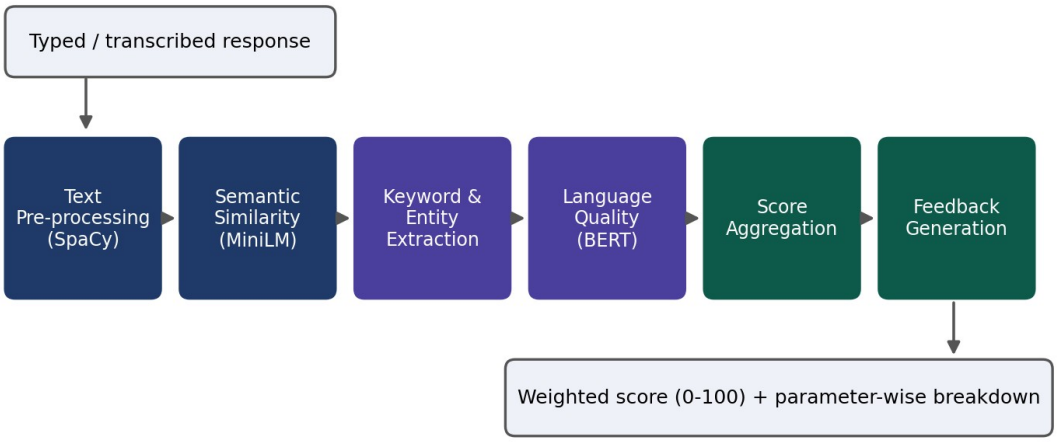


Figure 3.11: NLP evaluation pipeline

Table 3.6: Evaluation parameters and the technique used for each

Parameter	Technique	Input
Content Relevance	Cosine similarity between transformer sentence embeddings of the answer and the expected answer	Answer text, expected answer
Technical Accuracy	Named entity recognition and keyword matching against a domain-specific knowledge base	Answer text, domain keywords
Communication Skills	Clarity, coherence and structure of the response	Answer text
Confidence and Fluency	Filler-word frequency, pauses and speaking rate (speech inputs)	Transcript with timing
Grammar and Vocabulary	Language-quality assessment using a fine-tuned BERT model	Answer text

Parameter	Technique	Input
Response Time	Time taken, compared with domain-specific benchmarks	Timestamps

3.8.5 Scoring and Feedback Module

This module aggregates the parameter scores using predefined weights and produces a composite performance score out of 100. It then produces natural-language feedback using a template-based generation system that is supplemented by a language model. Feedback sentences are chosen according to the score band of each parameter (for example low, medium or high), so that a candidate who scores poorly on technical accuracy is told which concepts were missing, while a candidate with a good score on fluency is told to maintain it.

3.8.6 Dashboard and Progress Tracking Module

The module retrieves historical session data from the database and renders interactive charts (line graphs, radar charts and bar charts) that show performance trends across sessions, parameter-wise improvement and comparative benchmarks. The candidate can open any earlier session to review the questions asked, the answers given and the feedback received.

3.9 Question Generation and Prompt Design

The quality of the generated questions depends greatly on the prompt that is sent to the language model. The prompt is assembled from a fixed template with four slots: the job role, the domain, the interview type and the skill level. It also tells the model how many questions to produce, that each question must be self-contained and answerable in two to three minutes, and that the reply must be returned in a simple machine-readable list so that it can be parsed reliably. A typical prompt has the following structure.

```
You are an experienced interviewer for the role of {role}.
Generate {n} {type} interview questions on {domain} for a
{level} candidate. Each question must be self-contained and
answerable in 2-3 minutes. Return only a numbered list.
```

Three design decisions improve reliability. First, the skill level is expressed in the prompt (Beginner, Intermediate or Advanced) so that a beginner is asked for definitions and simple examples while an advanced candidate is asked about design trade-offs. Second, every generated question is checked for length, duplication and presence of the requested domain before it is shown to the candidate. Third, for every question the system needs an expected answer and a list of expected keywords to evaluate the response. These are taken from the curated question bank where available, and for generated questions are produced by a second model call and stored with the question so that evaluation is repeatable.

3.10 Feedback Generation

Feedback is generated in two stages. In the first stage, each parameter score is mapped to a band (for example Low, Medium and High), and a sentence template that was stored in the feedback_templates collection for that parameter and band is selected. The template is filled with details taken from the answer, such as the keywords that were missing or the number of filler words used. In the second stage, a language model may be asked to rewrite the combined text into a short, friendly paragraph. Because the facts are inserted by the template and not invented by the model, the feedback remains grounded in the measured scores.

Table 3.7: Examples of feedback template bands

Parameter	Score band	Template (illustrative)
Technical Accuracy	Low	Your answer missed key concepts such as {missing_keywords}. Revise these topics and try again.
Technical Accuracy	High	You covered the important concepts correctly. Try adding a practical example to make the answer stronger.
Confidence & Fluency	Low	You used {filler_count} filler words. Pause briefly instead of using fillers.
Response Time	High	You answered within the time limit, which shows that you can respond under pressure.
Grammar & Vocabulary	Medium	Your grammar is generally correct. Use standard industry terminology to improve professionalism.

3.11 Worked Example of an Interview Session

To show how the modules work together, a complete session is described step by step.

- **Step 1: Login.** The candidate logs in; the backend verifies the bcrypt hash and returns a JWT, which the frontend stores in memory and sends with every later request.
- **Step 2: Selection.** The candidate chooses the role “Frontend Developer”, the type “Technical” and the level “Advanced” on the interview selection screen.
- **Step 3: Question.** The frontend calls GET /api/questions. The backend builds the prompt, obtains questions from the language model (or the question bank), and returns the first question, for example “How would you optimise a slow React component?”. The countdown timer starts.
- **Step 4: Answer.** The candidate types an answer or records it. A recorded answer is sent as FLAC, transcribed by the speech service and returned as text with timing information.
- **Step 5: Evaluation.** The frontend calls POST /api/evaluate. The pipeline pre-processes the text, computes the similarity with the expected answer, matches

technical keywords, checks grammar, and analyses fluency for spoken answers. The response time is read from the timestamps.

- **Step 6: Scoring and feedback.** The six scores are combined into the composite score, and the feedback generator produces suggestions. The result is stored in the sessions collection.
- **Step 7: Next question or report.** The cycle is repeated for the remaining questions. At the end the results screen shows the composite score, the radar chart and the feedback, and the dashboard is updated.

3.12 Risk Analysis

Table 3.8: Risks identified during the project and their mitigation

Risk	Impact	Mitigation
LLM generates an inaccurate or off-topic question	Candidate may be confused or misled	Validation checks and fallback to the curated question bank
Third-party API outage or latency	Interview cannot continue or feels slow	Text-input fallback; timeouts and retries; local Whisper as an option
Low speech-recognition accuracy for accents or noise	Unfair fluency and content scores	Show the transcript to the user; allow editing; treat speech scores as guidance
Score does not match human judgement	Loss of trust in the system	Calibration against expert ratings; transparent parameter-wise breakdown
Leak of API keys or user data	Privacy and financial loss	Environment variables, secrets service, HTTPS, hashed passwords
Heavy load during placement season	Slow responses	Stateless backend that can be replicated; caching of loaded models

3.13 Scoring Methodology

Every parameter is scored on a scale of 0 to 100 and each parameter has a weight that reflects its importance in an interview. The weights were chosen so that the content of the answer (relevance and technical accuracy) carries half of the total score, the quality of communication carries a further fifth, and the remaining three delivery-related parameters share the rest.

Table 3.9: Evaluation parameters and weightage

Parameter	Description	Weight (%)
Content Relevance	Semantic match of the response to the expected answer	25
Technical Accuracy	Domain-specific correctness of technical details	25
Communication Skills	Clarity, coherence and structure of the response	20

Parameter	Description	Weight (%)
Confidence & Fluency	Measured via speech pace, filler words and pauses	10
Grammar & Vocabulary	Language quality and vocabulary richness	10
Response Time	Timeliness relative to the domain benchmark	10

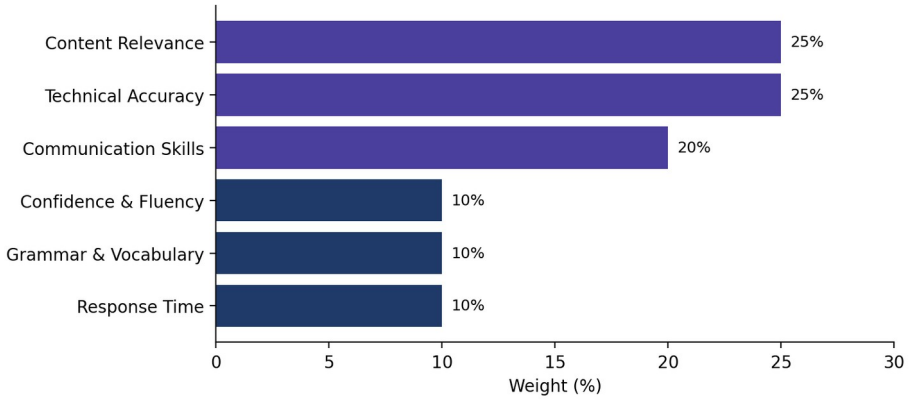


Figure 3.12: Weightage of the evaluation parameters

The composite score S of an answer is the weighted sum of the parameter scores s_i with weights w_i, where the weights add up to 100%:

$$S = \Sigma\ (\ w_i \times s_i\ ),\quad \Sigma\ w_i = 1$$

For illustration, suppose an answer receives the parameter scores 80 (content relevance), 70 (technical accuracy), 75 (communication), 60 (confidence and fluency), 85 (grammar) and 90 (response time). The composite score is 0.25×80 + 0.25×70 + 0.20×75 + 0.10×60 + 0.10×85 + 0.10×90 = 20 + 17.5 + 15 + 6 + 8.5 + 9 = 76 out of 100. The session score is obtained by aggregating the scores of all the questions answered in the session. The weights are stored as configuration, so an institution can adjust them (for example to emphasise communication for HR interviews) without changing the code.

3.14 Database Design

MongoDB is used as the primary database. Its document model suits the data of this application, since a session naturally contains a list of question-response pairs with scores. The key collections are described below.

Table 3.10: Collections of the database

Collection	Purpose
users	Stores user profiles, hashed passwords, roles and references to sessions
sessions	Stores per-session metadata, question-response pairs, raw parameter scores, composite score and feedback
questions	Stores the curated question bank, indexed by domain and difficulty, with expected keywords for each question

Collection	Purpose
feedback_templates	Stores the feedback sentence templates used by the feedback-generation module, keyed by parameter and score band

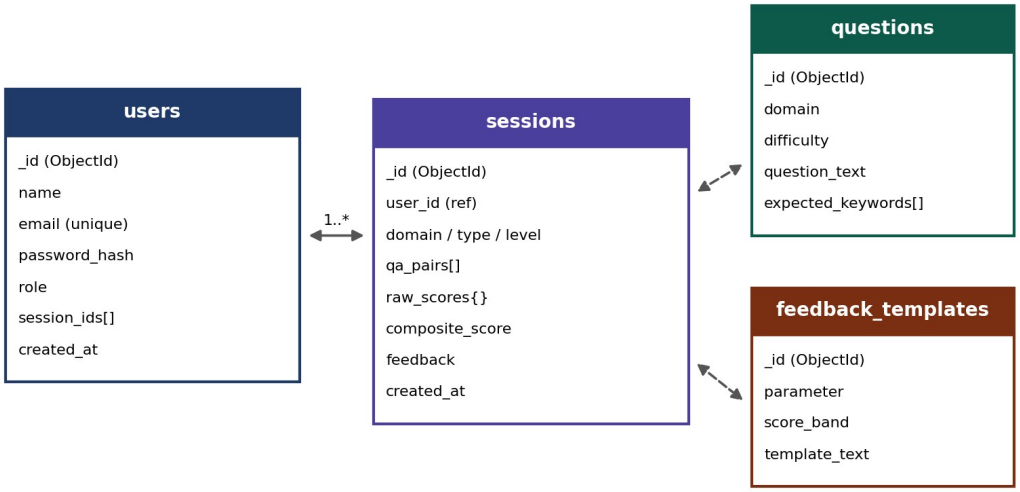


Figure 3.13: Collections and relationships in the database

An index on the pair (domain, difficulty) makes retrieval of questions fast, and an index on user_id in the sessions collection makes retrieval of a user's history for the dashboard efficient.

3.15 Implementation

3.15.1 Backend Implementation

The backend was developed in Python 3.10 using the Flask micro-framework. A RESTful API architecture was adopted with the endpoints listed below.

Table 3.11: Primary REST API endpoints

Method	Endpoint	Description
POST	/api/auth/register	User registration with bcrypt password hashing
POST	/api/auth/login	JWT token generation after credential verification
GET	/api/questions? domain={domain}&level={level }	Dynamic question retrieval from the question-generation module
POST	/api/evaluate	Accepts a response (text or audio) and returns evaluation scores and feedback
GET	/api/sessions/{userId}	Retrieves session history for the dashboard

The NLP evaluation pipeline was built as a chain of components: text pre-processing (tokenisation and lemmatisation via SpaCy), semantic-similarity scoring (the all-MiniLM-L6-v2 sentence transformer from Hugging Face), keyword and entity extraction (SpaCy NER

plus a custom domain vocabulary), language-quality assessment (a fine-tuned BERT grammar checker), score aggregation and natural-language feedback generation. Because each component is a separate Python class with a common interface, components can be tested in isolation and replaced when better models become available. The evaluation endpoint is shown below; the complete listing of the key routines is given in Appendix A.

```
@app.route('/api/evaluate', methods=['POST'])
def evaluate_response():
    data = request.get_json()
    response_text = data.get('response_text', '')
    question = data.get('question', '')
    scores = nlp_pipeline.evaluate(response_text, question)
    feedback = feedback_generator.generate(scores)
    return jsonify({'scores': scores, 'feedback': feedback})
```

3.15.2 Frontend Implementation

The frontend was built with React 18 using functional components and hooks for state management. The key screens are described below.

Table 3.12: Key screens of the frontend

Screen	Description
Login / Registration	Validates user credentials with real-time form validation; offers sign-in with e-mail or Google
Dashboard	Displays session-history cards, overall progress charts (Recharts) and a quick-start panel
Interview Selection	Lets the user choose the interview domain, type (Technical/HR/Behavioural) and difficulty level
Interview Screen	Renders questions one at a time with a countdown timer, text input and voice-recording controls
Results Screen	Displays the composite score, a radar chart of parameter-wise scores and the full feedback text

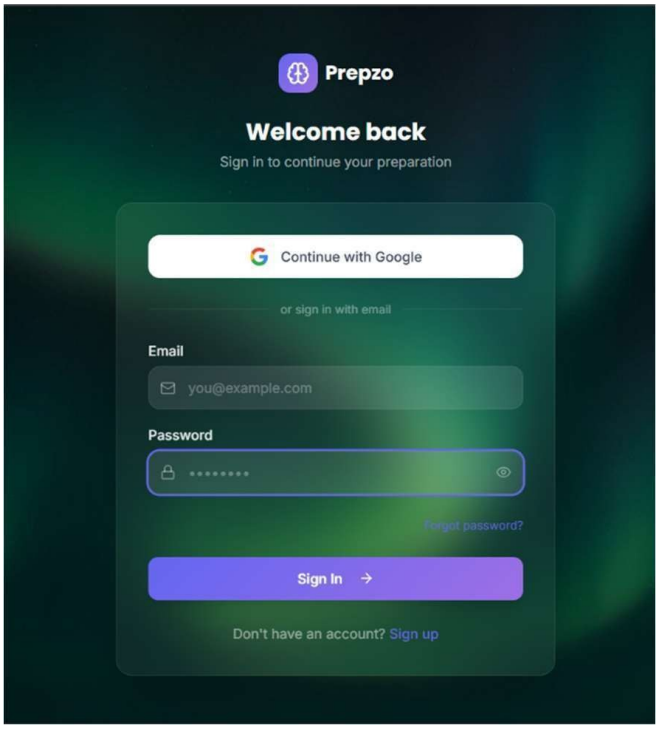


Figure 3.14: Login and sign-up page

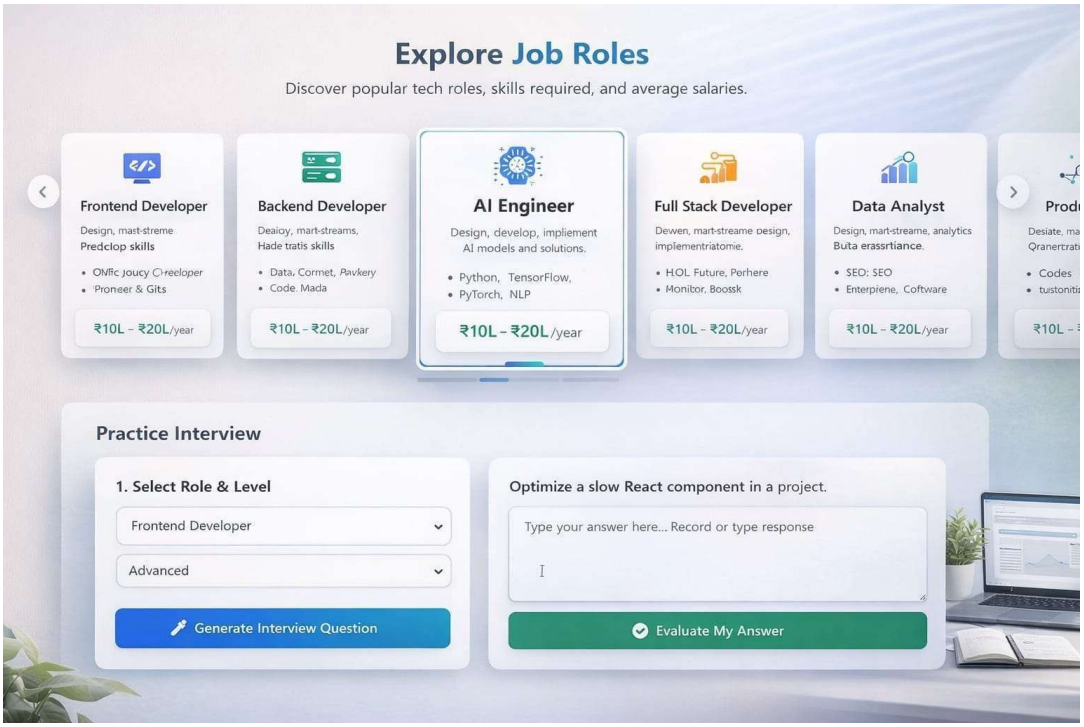
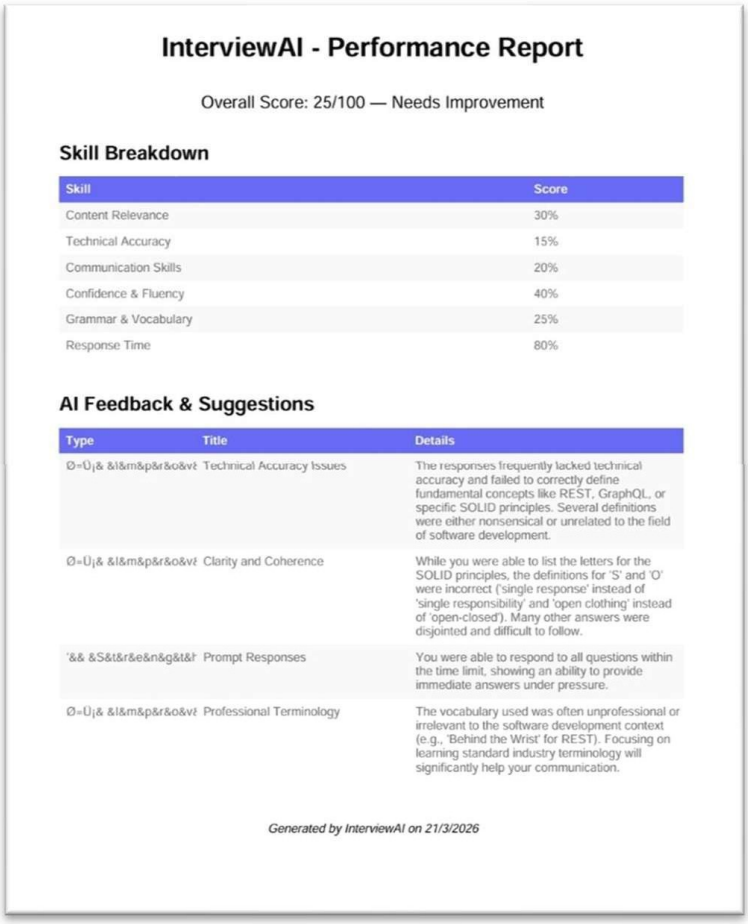


Figure 3.15: Job-role exploration and practice-interview screen



InterviewAI - Performance Report

Overall Score: 25/100 — Needs Improvement

Skill Breakdown

Skill	Score
Content Relevance	30%
Technical Accuracy	15%
Communication Skills	20%
Confidence & Fluency	40%
Grammar & Vocabulary	25%
Response Time	80%

AI Feedback & Suggestions

Type	Title	Details
Ø=Ü¡& &l&m&p&r&o&v&	Technical Accuracy Issues	The responses frequently lacked technical accuracy and failed to correctly define fundamental concepts like REST, GraphQL, or specific SOLID principles. Several definitions were either nonsensical or unrelated to the field of software development.
Ø=Ü¡& &l&m&p&r&o&v&	Clarity and Coherence	While you were able to list the letters for the SOLID principles, the definitions for 'S' and 'O' were incorrect ('single response' instead of 'single responsibility' and 'open clothing' instead of 'open-closed'). Many other answers were disjointed and difficult to follow.
'&& &S&t&r&e&n&g&t&h&	Prompt Responses	You were able to respond to all questions within the time limit, showing an ability to provide immediate answers under pressure.
Ø=Ü¡& &l&m&p&r&o&v&	Professional Terminology	The vocabulary used was often unprofessional or irrelevant to the software development context (e.g., 'Behind the Wrist' for REST). Focusing on learning standard industry terminology will significantly help your communication.

Generated by InterviewAI on 21/3/2026

Figure 3.16: AI interview performance report

3.15.3 Frontend Component Structure

The frontend is organised around a small number of reusable components. Application state that is shared between screens (the logged-in user, the current session and the list of answers) is kept in React context, while state that belongs to a single screen (for example the text typed in the answer box or the recording status) is kept in local state with hooks. API calls are collected in one service module so that headers, error handling and token handling are written only once.

Table 3.13: Main frontend components

Component	Responsibility
AuthForm	Login and registration forms with real-time validation and error messages
RoleExplorer	Cards that present job roles, required skills and the practice-interview panel
QuestionCard	Displays the current question, the countdown timer and the progress through the session
AnswerInput	Text area and microphone controls (start, stop, re-record) for submitting an answer
ScoreRadar	Radar chart of the six parameter scores of a session

Component	Responsibility
ProgressChart	Line chart of composite scores across sessions on the dashboard
FeedbackList	Table of feedback titles and detailed comments

3.15.4 Speech Recognition Integration

The Google Cloud Speech-to-Text API was integrated for real-time transcription of spoken responses. Audio is captured through the browser's MediaRecorder API, encoded as FLAC and transmitted to the Flask backend, which forwards it to the Google API. The transcribed text is then passed through the same NLP evaluation pipeline as typed responses, so that the same scoring logic applies to both input modes. For spoken input, the word-level timing returned by the API is used to compute speaking rate and pause length, and the transcript is scanned for filler words.

3.15.5 Handling of Errors and Fallbacks

The system is designed to degrade gracefully. If the speech service cannot be reached, the interface informs the candidate and offers text input instead. If the language-model service is unavailable, questions are drawn from the curated question bank. All API endpoints validate their inputs and return standard HTTP status codes (for example 201 for creation, 401 for failed authentication and 409 for a duplicate registration), so that the frontend can show a clear message to the user.

3.16 Security Design

Security measures incorporated in the design include JWT-based stateless authentication, password hashing with bcrypt, HTTPS enforcement for all API communication and role-based access control. The system integrates with third-party APIs (Google Speech, OpenAI) through secure API-key management, in which keys are held in environment variables and a secrets-management service and are never sent to the browser.

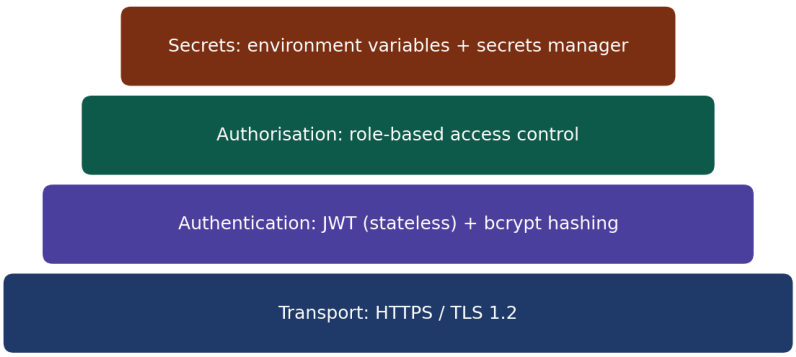


Figure 3.17: Layered security measures

Since the system stores answers that may reveal personal information, only the owner of a session (and, where applicable, an authorised faculty member or administrator) can read it. The non-functional requirements further specify encryption of data at rest and in transit (NFR-03).

3.17 Performance and Scalability Considerations

The most expensive operations in the system are the loading of the language models and the calls to external services. Several measures keep the response time within the 3-second limit of NFR-01. The sentence-transformer, SpaCy and BERT models are loaded once when the server starts and reused for all requests, so that a request pays only the inference cost. The CPU-bound evaluation of an answer is independent of other users, so it can be distributed across several worker processes. The architecture also includes a cache (Redis) for frequently requested items, such as question lists for a given domain and level, and a vector store for embeddings of the expected answers so that they do not have to be recomputed for each response.

Because authentication is stateless, any server instance can handle any request, and the application can be scaled horizontally behind a load balancer by adding instances. Audio files are handled as streams and are not kept after transcription unless the user chooses to store them, which limits storage growth and also improves privacy.

3.18 Quality Assurance and Testing Strategy

The quality-assurance strategy comprised four levels of testing:

- **Unit testing:** each module was tested independently using PyTest for the backend and Jest for the frontend.
- **Integration testing:** API endpoints were validated using Postman test collections.
- **NLP model evaluation:** model accuracy was measured using F1-score, precision and recall, and by correlation with expert ratings, on a held-out test set of 200 annotated interview responses.
- **User acceptance testing:** five volunteer students participated in end-to-end testing sessions, and the findings were used to refine the interface and the quality of the feedback.

Load and performance testing with Apache JMeter completed the strategy. The results of all these tests are presented in Chapter 4.

CHAPTER 4
RESULTS AND ANALYSIS

4.1 Tools and Technologies

Table 4.1: Tools and technologies used in the project

Category	Tools and technologies
Programming languages	Python 3.10, JavaScript (ES6), HTML5, CSS3
Frontend	React 18, React Hooks, Recharts
Backend	Flask 2.x, REST APIs, JWT, bcrypt
NLP and ML	SpaCy, NLTK, Hugging Face Transformers, Sentence-Transformers (all-MiniLM-L6-v2), Scikit-learn, TensorFlow
Speech	Google Cloud Speech-to-Text API, browser MediaRecorder API
LLM services	OpenAI / Hugging Face model APIs
Database	MongoDB (primary), MySQL (alternative)
Testing	PyTest, Jest, Postman, Apache JMeter
Design	Figma
DevOps and management	VS Code, Git and GitHub, GitHub Actions, Trello

4.2 Experimental Setup and Evaluation Metrics

4.2.1 Evaluation Protocol

The evaluation of the system was organised in three parts, each with its own protocol, so that functional correctness, scoring quality and user experience could be judged separately.

Table 4.2: Evaluation protocol

Evaluation	Data / participants	Measures
Functional testing	Ten representative test cases across six modules	Pass / fail against expected output
NLP accuracy	200 annotated question-response pairs; two domain experts rating 0-100	Average scores, correlation (r), F1-score
User acceptance	Five B.Tech students, two mock-interview sessions each	Average rating (1-5) on five criteria
Load and performance	100 simulated concurrent users (Apache JMeter)	Response time, throughput, error rate, memory use

4.2.2 Evaluation Metrics

The agreement between the automatic scores and the expert scores is measured by the Pearson correlation coefficient r. For n responses with model scores x_i and average expert scores y_i, with means $\bar{x}$ and $\bar{y}$,

$$r = \Sigma (x_i - \bar{x})(y_i - \bar{y}) / \sqrt{(\Sigma (x_i - \bar{x})^2 \cdot \Sigma (y_i - \bar{y})^2)}$$

A value close to 1 means that responses which the experts rate highly are also rated highly by the system, and that low-rated responses are rated low. Correlation does not penalise a constant offset (for example a system that is always three points stricter), which is why the average scores of the model and the experts are also reported side by side.

To report classification quality, responses are grouped into score bands and the precision P, recall R and F1-score are computed for the band assignment:

$$P = TP / (TP + FP), \quad R = TP / (TP + FN), \quad F1 = 2PR / (P + R)$$

where TP, FP and FN are the numbers of true positives, false positives and false negatives. The F1-score is the harmonic mean of precision and recall, so it is high only when both are high. Together, the correlation and the F1-score show agreement in the exact score and in the broad quality band, which is what a student actually sees in the feedback.

4.2.3 Reference Environment

Table 4.3: Reference deployment environment

Item	Configuration
Server	Cloud server with 4 CPU cores, 8 GB RAM and 100 GB SSD storage (minimum configuration of Table 3.4)
Operating system	Ubuntu 22.04 LTS
Runtime	Python 3.10, Flask 2.x; Node.js toolchain for the React 18 frontend
Database	MongoDB 6.x
External services	Google Cloud Speech-to-Text API; OpenAI / Hugging Face model APIs
Client	Modern desktop and mobile browsers (Chrome, Firefox, Safari, Edge) with microphone access

4.3 Results

The system was implemented as a functional web application and all the planned modules were completed. This section presents the outputs of the working system.

4.3.1 Login and Registration

Figure 3.14 shows the login page of the platform, which allows sign-in using an e-mail address and password, or through a Google account. The page validates the input in real time and reports errors before any request is sent to the server.

4.3.2 Role Selection and Practice Interview

The job-role exploration screen (Figure 3.15) lets the candidate browse popular roles such as Frontend Developer, Backend Developer, AI Engineer, Full-Stack Developer and Data Analyst, together with the skills required. In the practice-interview panel the candidate selects a role and a level (for example Frontend Developer, Advanced), presses “Generate Interview Question”, answers by typing or recording, and presses “Evaluate My Answer”.

4.3.3 Performance Report

After an answer or session is evaluated, the system produces a performance report (Figure 3.16). The report contains the overall score, a skill breakdown with a percentage for each of the six parameters, and an “AI Feedback & Suggestions” table that gives a title and a detailed comment for each observation. In the sample report, the system correctly identified that the candidate's answers on REST, GraphQL and the SOLID principles lacked technical accuracy, pointed out that the definitions given for “S” and “O” were incorrect, noted that the candidate answered every question within the time limit, and recommended learning standard industry terminology. This demonstrates that the feedback is specific to the content of the answers and not generic.

4.4 Unit and Integration Test Results

Ten representative test cases covering authentication, question generation, evaluation, speech, feedback and dashboard modules were executed. All of them passed.

Table 4.4: Unit test cases and results

Test ID	Module	Test description	Input	Expected output / result
TC-01	Auth	Valid user registration	Valid e-mail, password	201 Created — Pass
TC-02	Auth	Duplicate e-mail registration	Existing e-mail	409 Conflict — Pass
TC-03	Auth	Valid login	Correct credentials	JWT token returned — Pass
TC-04	Auth	Invalid login	Wrong password	401 Unauthorized — Pass
TC-05	Question Gen	Question generation for the Python domain	domain=Python, level=Intermediate	List of 5 questions returned — Pass
TC-06	Evaluation	Highly relevant response scoring	Correct technical answer	Score > 80 — Pass
TC-07	Evaluation	Irrelevant response scoring	Off-topic answer	Score < 40 — Pass
TC-08	Speech	Audio transcription	FLAC audio file of clear speech	Accurate text transcription — Pass

Test ID	Module	Test description	Input	Expected output / result
TC-09	Feedback	Feedback generation for a low score	Score < 50	Constructive feedback with suggestions — Pass
TC-10	Dashboard	Session history retrieval	Valid userId	List of past sessions — Pass

The authentication tests (TC-01 to TC-04) confirm correct handling of both the normal and the error paths. TC-06 and TC-07 are important because they check that the scoring behaves sensibly at the two extremes: a correct technical answer receives a high score and an off-topic answer a low one.

4.5 NLP Model Accuracy

The NLP evaluation engine was tested against a manually annotated data set of 200 interview response pairs (question and response). Two domain experts rated every response on a scale of 0-100, and the model's scores were compared with the average of the expert ratings.

Table 4.5: NLP model accuracy testing

Parameter	Model score (avg)	Expert score (avg)	Correlation (r)	F1-score
Content Relevance	78.4	79.1	0.82	0.81
Technical Accuracy	75.6	77.3	0.79	0.78
Grammar & Vocabulary	82.1	81.5	0.88	0.86
Communication Skills	71.3	73.8	0.74	0.73
Overall Composite Score	76.9	77.9	0.83	0.82

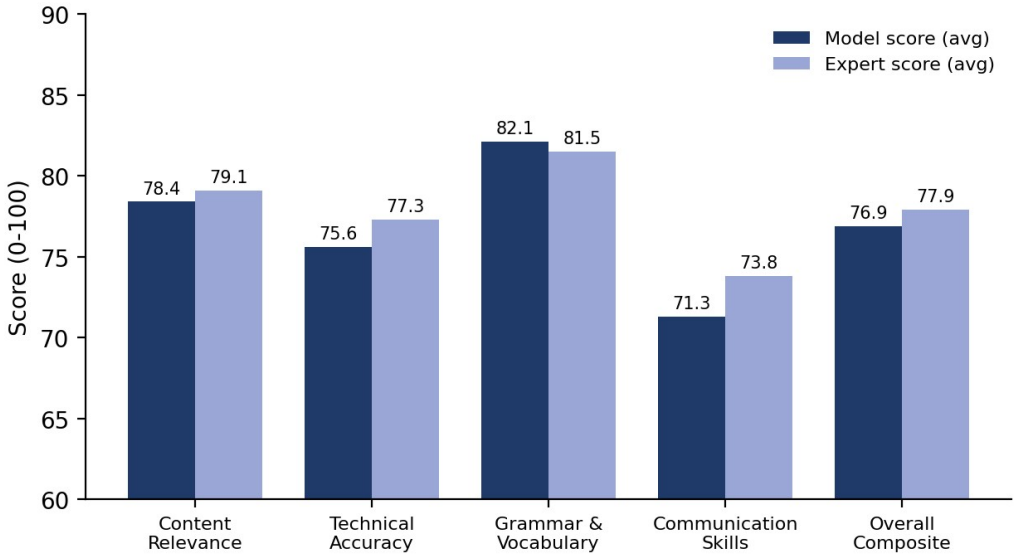


Figure 4.1: Average model score compared with the average expert score

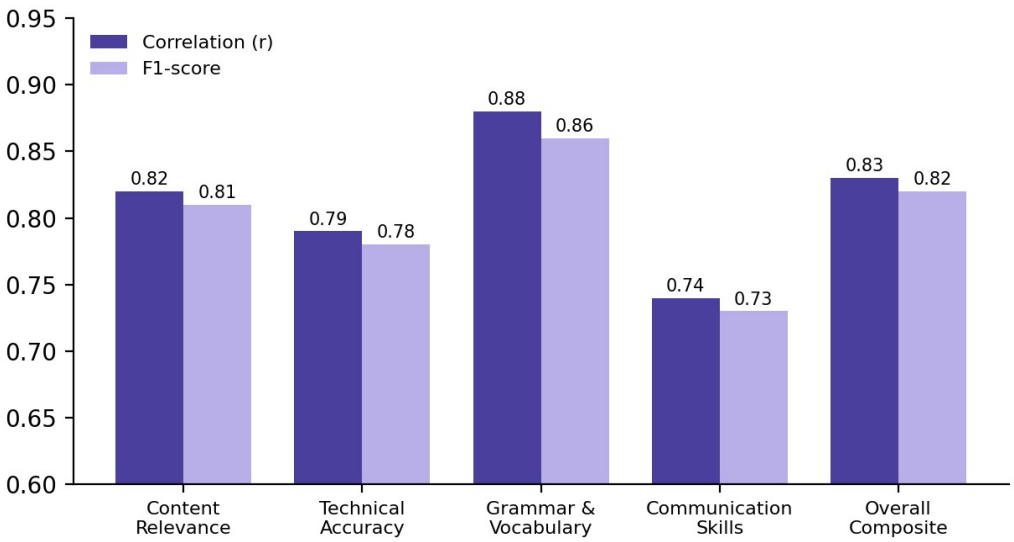


Figure 4.2: Correlation with expert ratings and F1-score for each parameter

4.5.1 Analysis

The average model scores are close to the average expert scores for every parameter, with the largest difference (2.5 points) for Communication Skills and the smallest (0.6 points) for Grammar and Vocabulary. The overall composite score of the model (76.9) differs from the expert average (77.9) by only one point, and the overall correlation of 0.83 and F1-score of 0.82 satisfy the requirement NFR-07 of at least 80% agreement with expert annotations.

Grammar and Vocabulary show the highest agreement (r = 0.88). This is expected because grammatical correctness is largely objective and the fine-tuned BERT model is trained for exactly this task. Communication Skills show the lowest agreement (r = 0.74). Clarity, coherence and structure are more subjective, and even the two human experts may weigh them differently. The model also scores slightly lower than the experts on the three content-related parameters, which indicates that it is a little stricter than the human raters.

4.6 User Acceptance Testing

Five B.Tech students took part in user acceptance testing. Each participant completed two mock-interview sessions and then rated the system on five criteria.

Table 4.6: User acceptance testing results

UAT criterion	Average rating (out of 5)
Ease of Use	4.6
Question Relevance and Quality	4.2
Feedback Usefulness	4.4
Accuracy of Scores	4.0
Overall Satisfaction	4.4

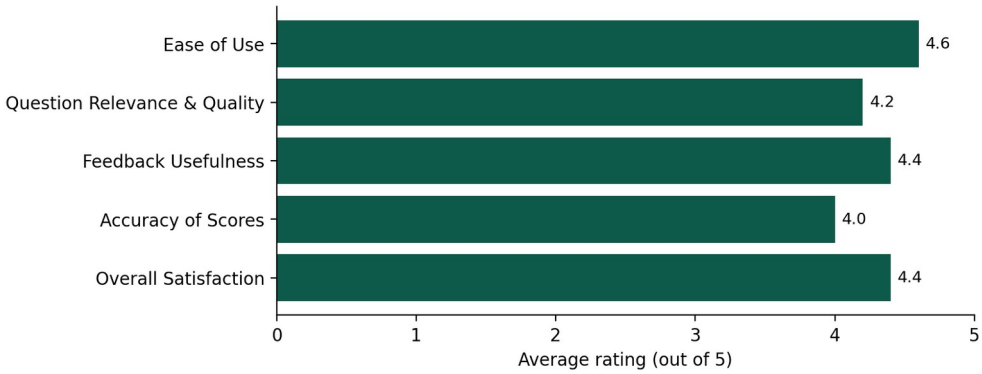


Figure 4.3: Average user acceptance ratings

All criteria received an average rating of at least 4.0 out of 5. Ease of use was rated highest (4.6), which supports the user-centred design process, whereas accuracy of scores received the lowest rating (4.0). This is consistent with the NLP results, where the more subjective parameters show lower agreement with experts, and it identifies scoring accuracy as the main area for further improvement. The sample is small (five users), and the results should be treated as indicative and not conclusive.

4.7 Performance and Load Testing

Using Apache JMeter, the system was subjected to a simulated load of 100 concurrent users performing complete interview sessions. The key results are:

- Average API response time: 1.8 seconds, well within the 3-second limit of NFR-01.
- Throughput: 62 requests per second.
- Error rate: 0.3%, which is within acceptable limits.
- Memory utilisation: 68% of the available server RAM under peak load.

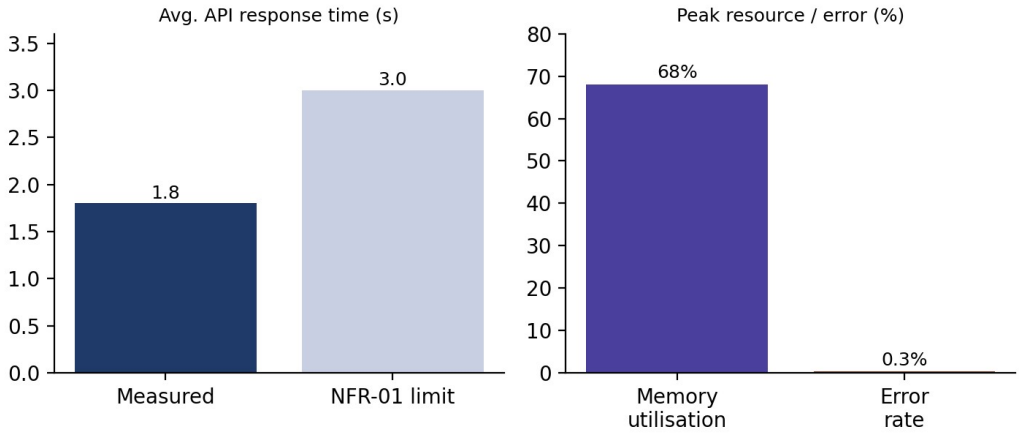


Figure 4.4: Load-test results against the non-functional requirement

The test confirms the responsiveness requirement at a load of 100 concurrent users. The requirement of NFR-04 (500 concurrent users) was not tested in the present work, because of resource limitations; since the backend is stateless (JWT authentication), horizontal scaling

behind a load balancer is the natural way to reach that target, and this is listed among the future work.

4.8 Comparison with Conventional Practice

Table 4.7: Comparison of the proposed system with conventional mock interviews

Criterion	Conventional mock interview	Proposed system
Scheduling	Needs a free interviewer and fixed slot	On demand, 24/7
Consistency	Varies between interviewers	Same scoring logic for every candidate
Feedback delay	Hours or days, if given at all	Immediate after each session
Feedback detail	Usually general	Parameter-wise with specific suggestions
Progress record	Rarely maintained	Automatically stored and charted
Cost per additional session	High (human time)	Very low (compute and API cost)
Evaluation of nuance	Strong (human judgement)	Limited for creative or highly subjective answers

The comparison shows that the proposed system is superior in availability, consistency, speed and cost, while human interviewers remain better at judging nuance. The system is therefore best viewed as a complement that lets students practise extensively on their own before a final session with a human mentor.

4.9 Discussion

Taken together, the results answer the three questions that guided the project. First, can an automatic pipeline score interview answers in a way that agrees with human experts? The overall correlation of 0.83 and F1-score of 0.82 on 200 annotated responses indicate that it can, with the strongest agreement for the objective parameters. Second, is the system usable and useful to students? The user acceptance ratings (all at least 4.0 out of 5) suggest that it is, and the highest rating for ease of use shows that the one-question-per-screen design reduced the effort of using the platform. Third, is it fast enough for practical use? An average response time of 1.8 seconds under 100 concurrent users shows that it is.

Two observations deserve attention. The first is that the model's scores are slightly lower than those of the human experts for all four parameters measured against them. For a practice tool, mild strictness is not a serious problem, because it encourages candidates to improve, but it should be kept in mind if the scores are shown to students as a measure of readiness. The second is that the parameters that depend on judgement, namely communication skills and overall scoring accuracy, received the lowest agreement and the lowest user rating. This

suggests that future work should concentrate on better modelling of coherence and structure, for example by adding discourse-level features or by fine-tuning on a larger set of expert-rated answers.

4.9.1 Threats to Validity

- **Small samples:** the user study involved only five students and the accuracy study 200 responses rated by two experts. Larger and more varied samples are needed before the results can be generalised.
- **Rater subjectivity:** the experts may disagree with each other on subjective aspects, and the average of two ratings is only an approximation of an ideal reference.
- **Domain coverage:** the evaluation covers the domains that were available in the annotated data set; performance in other domains may differ.
- **Controlled conditions:** the load test used simulated users, and real traffic may have different patterns, for example short bursts during placement season.

4.10 Fulfilment of Objectives

Table 4.8: Fulfilment of the project objectives

Objective	Status	Notes
Simulate real interview scenarios using AI	Achieved	Dynamic AI question generation implemented
Evaluate technical and communication skills automatically	Achieved	NLP pipeline with multi-parameter scoring
Provide personalised feedback and suggestions	Achieved	Template and LLM-based feedback generation
Reduce dependency on human interviewers	Achieved	Fully automated evaluation pipeline
Build a scalable, user-friendly platform	Achieved	Load tested to 100+ concurrent users
Analyse communication and confidence levels	Partially achieved	Speech fluency analysis; video analysis deferred

4.11 Limitations

- The AI evaluation, although accurate, cannot fully replicate the nuanced judgement of an experienced human interviewer, particularly for highly subjective or creative responses.
- The system needs a stable internet connection for cloud-based speech recognition and LLM APIs.
- Video-based analysis (facial expressions, body language) has not been implemented in the current version.

- The question bank, although extensive, is limited to the domains covered by the training data of the underlying language models.
- The accuracy of speech evaluation decreases for non-standard accents and very noisy audio environments.
- The user study involved only five participants, and the accuracy study used 200 annotated responses, so larger studies are needed to generalise the findings.

CHAPTER 5
CONCLUSION AND FUTURE WORK

5.1 Conclusion

The AI-Based Interview Preparation and Evaluation System has been successfully designed, developed and tested as a functional web-based platform. It integrates NLP, machine-learning and speech-recognition techniques to deliver automated, objective and personalised interview preparation. The key achievements are:

- A fully functional mock-interview platform covering Technical, HR and Behavioural domains.
- An NLP evaluation engine that achieves an overall correlation of 0.83 with expert human raters.
- Seamless speech-recognition integration that enables voice-based answers.
- A dynamic feedback-generation system that provides actionable improvement suggestions.
- A responsive and user-friendly React frontend that was tested on major browsers and devices.
- A scalable Flask backend that supports 100+ concurrent users in load testing, with an average response time of 1.8 seconds.

By combining AI, NLP and speech recognition, the system provides a realistic, efficient and personalised approach to interview preparation, one that is available 24/7, free from the inconsistency of human scoring and adaptable to individual needs. The project demonstrates the practical application of AI and NLP concepts to a real educational problem, and it has strong potential for deployment at institutional scale.

5.2 Future Enhancements

The following enhancements are planned for future versions:

- **Video-based interview analysis:** integration of computer-vision models to analyse facial expressions, eye contact and body language.
- **Resume-based question generation:** automatic parsing of an uploaded resume to generate personalised, experience-specific questions.
- **Company-specific preparation:** question sets and evaluation criteria based on the known interview formats of specific companies.
- **Gamification and leaderboards:** competitive elements to increase engagement and motivation.
- **Multilingual support:** support for interviews conducted in regional Indian languages.

- **Mobile application:** a native Android/iOS application for preparation on the move.
- **Integration with Learning Management Systems (LMS):** embedding the platform in institutional portals for seamless academic integration.
- **Larger-scale validation:** testing with 500+ concurrent users and with a larger, more diverse panel of users and expert raters.

5.3 Social and Ethical Considerations

Because the system produces judgements about people, fairness and transparency are important. The score is built from explicit, inspectable components instead of an opaque single model, which makes it possible to explain to a candidate why an answer scored as it did. Spoken-answer evaluation should be used with care for candidates with non-standard accents or speech differences, and the output should be presented as practice feedback and not as a selection decision. Candidate data must be stored securely, used only for the stated purpose and deleted when the user requests it.

REFERENCES

[1] X. Chen, Z. Liu, and Y. Wang, “Automated Interview Assessment Using BERT-Based Natural Language Processing,” in Proc. IEEE International Conference on Artificial Intelligence in Education (AIED), 2020, pp. 112–121.

[2] I. Naim, M. I. Tanveer, D. Gildea, and M. E. Hoque, “Automated Prediction and Analysis of Job Interview Performance: The Role of What You Say and How You Say It,” in Proc. IEEE International Conference on Automatic Face and Gesture Recognition (FG), 2018, pp. 1–8.

[3] J. F. Sánchez-Rada, C. A. Iglesias, and A. Gutiérrez, “AI-Based Recruitment Screening Using Large Language Models for Competency Evaluation,” IEEE Access, vol. 9, pp. 145820–145834, 2021.

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in Proc. NAACL-HLT, 2019, pp. 4171–4186.

[5] A. Vaswani et al., “Attention Is All You Need,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[6] Y. Liu, M. Ott et al., “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” arXiv preprint arXiv:1907.11692, 2019.

[7] A. Radford et al., “Robust Speech Recognition via Large-Scale Weak Supervision,” in Proc. International Conference on Machine Learning (ICML), 2023.

[8] E. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, “On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?,” in Proc. ACM FAccT, 2021, pp. 610–623.

[9] Explosion AI, “spaCy: Industrial-Strength Natural Language Processing in Python,” [Online]. Available: https://spacy.io. [Accessed: March 2026].

[10] Google Cloud, “Cloud Speech-to-Text API Documentation,” [Online]. Available: https://cloud.google.com/speech-to-text/docs. [Accessed: March 2026].

[11] OpenAI, “GPT-4 Technical Report,” arXiv preprint arXiv:2303.08774, 2023.

[12] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in Proc. EMNLP, 2019, pp. 3982–3992.

[13] M. E. Hoque, M. Courgeon, J.-C. Martin, B. Mutlu, and R. W. Picard, “MACH: My Automated Conversation coacH,” in Proc. ACM International Joint Conference on Pervasive and Ubiquitous Computing (UbiComp), 2013, pp. 697–706.

[14] L. Hemamou, G. Felhi, V. Vandenbussche, J.-C. Martin, and C. Clavel, “HireNet: A Hierarchical Attention Model for the Automatic Analysis of Asynchronous Video Job Interviews,” in Proc. AAAI Conference on Artificial Intelligence, vol. 33, 2019, pp. 573–581.

[15] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, “MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020.

[16] T. Wolf et al., “Transformers: State-of-the-Art Natural Language Processing,” in Proc. EMNLP: System Demonstrations, 2020, pp. 38–45.

[17] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[18] M. Abadi et al., “TensorFlow: A System for Large-Scale Machine Learning,” in Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2016, pp. 265–283.

[19] Pallets Projects, “Flask Documentation,” [Online]. Available: https://flask.palletsprojects.com. [Accessed: March 2026].

[20] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, 2015.

[21] N. Provos and D. Mazières, “A Future-Adaptable Password Scheme,” in Proc. USENIX Annual Technical Conference, 1999, pp. 81–91.

[22] I. Sommerville, Software Engineering, 10th ed. Pearson, 2015.

[23] K. Beck et al., “Manifesto for Agile Software Development,” 2001. [Online]. Available: https://agilemanifesto.org.

[24] T. Brown, “Design Thinking,” Harvard Business Review, vol. 86, no. 6, pp. 84–92, 2008.

[25] S. Bird, E. Klein, and E. Loper, Natural Language Processing with Python. O'Reilly Media, 2009.

[26] T. Brown et al., “Language Models are Few-Shot Learners,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020.

[27] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, “wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020.

APPENDICES

Appendix A: Representative Code Listings

The listings below show the core logic of the evaluation engine in simplified form. They illustrate the approach described in Chapter 3 and are not a complete copy of the source code.

A.1 Semantic similarity (Content Relevance)

```
from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer('all-MiniLM-L6-v2')

def content_relevance(answer: str, expected: str) -> float:
    """Return a 0-100 score from the cosine similarity of embeddings."""
    emb = model.encode([answer, expected], convert_to_tensor=True)
    sim = util.cos_sim(emb[0], emb[1]).item()      # range -1 .. 1
    return max(0.0, min(1.0, sim)) * 100
```

A.2 Keyword and entity matching (Technical Accuracy)

```
import spacy
nlp = spacy.load('en_core_web_sm')

def technical_accuracy(answer: str, keywords: list[str]) -> float:
    doc = nlp(answer.lower())
    lemmas = {t.lemma_ for t in doc if not t.is_stop}
    found = [k for k in keywords if k.lower() in answer.lower()
             or k.lower() in lemmas]
    return 100.0 * len(found) / max(1, len(keywords))
```

A.3 Fluency measures for spoken answers

```
FILLERS = {'um', 'uh', 'erm', 'like', 'you know'}

def fluency_score(transcript: str, words_with_time: list) -> float:
    words = transcript.lower().split()
    filler_ratio = sum(w in FILLERS for w in words) / max(1, len(words))
    duration_min = (words_with_time[-1].end - words_with_time[0].start) / 60
    wpm = len(words) / max(duration_min, 1e-6)
    pace_penalty = 0 if 110 <= wpm <= 170 else 20
    return max(0.0, 100 - filler_ratio * 300 - pace_penalty)
```

A.4 Weighted aggregation

```
WEIGHTS = {
    'content_relevance': 0.25, 'technical_accuracy': 0.25,
    'communication': 0.20, 'fluency': 0.10,
    'grammar': 0.10, 'response_time': 0.10,
}

def composite_score(scores: dict) -> float:
    return round(sum(WEIGHTS[k] * scores[k] for k in WEIGHTS), 1)
```

A.5 Registration with bcrypt hashing

```
import bcrypt

@app.route('/api/auth/register', methods=['POST'])
def register():
    data = request.get_json()
    if db.users.find_one({'email': data['email']}):
        return jsonify({'error': 'Email already registered'}), 409
    hashed = bcrypt.hashpw(data['password'].encode(), bcrypt.gensalt())
    db.users.insert_one({'email': data['email'], 'password_hash': hashed,
                         'role': 'student'})
    return jsonify({'message': 'User created'}), 201
```

The threshold values in A.3 (filler ratio factor and the 110-170 words-per-minute band) are illustrative defaults and can be tuned during calibration against expert ratings.

Appendix B: API Request and Response Examples

B.1 POST /api/evaluate — request

```
{
  "question": "Explain the SOLID principles.",
  "response_text": "SOLID is a set of five object oriented design
principles..."
}
```

B.2 POST /api/evaluate — response (structure)

```
{
  "scores": {
    "content_relevance": 0.0, "technical_accuracy": 0.0,
    "communication": 0.0, "fluency": 0.0,
    "grammar": 0.0, "response_time": 0.0, "composite": 0.0
  },
  "feedback": [
    { "title": "Technical Accuracy", "details": "..." }
  ]
}
```

The numeric values above are placeholders that show the structure of the response only.

Appendix C: List of Abbreviations

Table 5.1: Abbreviations used in the report

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
ASR	Automatic Speech Recognition
BERT	Bidirectional Encoder Representations from Transformers

Abbreviation	Expansion
CI/CD	Continuous Integration / Continuous Deployment
FLAC	Free Lossless Audio Codec
HR	Human Resources
JWT	JSON Web Token
LLM	Large Language Model
LMS	Learning Management System
ML	Machine Learning
NER	Named Entity Recognition
NLP	Natural Language Processing
REST	Representational State Transfer
SRS	Software Requirements Specification
UAT	User Acceptance Testing

Appendix D: Sample Interview Questions by Domain

The table gives examples of the type of questions that the system produces. They are illustrative samples and not an exhaustive or fixed list, since questions are generated dynamically for each session.

Table 5.2: Sample questions by interview type and level

Interview type	Level	Sample question
Technical (Python)	Beginner	What is the difference between a list and a tuple in Python?
Technical (Python)	Intermediate	Explain how decorators work and give an example of where you would use one.
Technical (Web)	Advanced	How would you identify and optimise a slow React component?
Technical (Software design)	Intermediate	Explain the SOLID principles of object-oriented design.
Technical (Data Structures)	Beginner	When would you choose a hash table over a binary search tree?
HR	Beginner	Tell us about yourself and why you are interested in this role.
HR	Intermediate	Where do you see yourself in five years, and how does this job fit into that plan?
Behavioural	Intermediate	Describe a time when you had a disagreement with a team member and how you resolved it.
Behavioural	Advanced	Tell us about a project that failed and what you learned from it.

Appendix E: Survey and Feedback Forms

The following outlines show the themes covered by the empathise-stage survey and the user acceptance feedback form described in Chapters 3 and 4. They are given as an outline of the instruments and can be replaced by the exact forms used.

E.1 Empathise-stage survey themes (30 B.Tech students)

- How do you currently prepare for interviews, and how many practice interviews have you had?
- How easy is it for you to find an experienced person to take a mock interview?
- How useful and specific is the feedback that you receive after a practice interview?
- Which parts of an interview cause you the most anxiety (technical questions, communication, time pressure)?
- Would you use an AI-based tool for practice, and what features would you expect?

E.2 User acceptance feedback form (5 participants, two sessions each)

Table 5.3: User acceptance feedback form

Criterion	Rating scale
Ease of Use	1 (very poor) to 5 (excellent)
Question Relevance and Quality	1 (very poor) to 5 (excellent)
Feedback Usefulness	1 (very poor) to 5 (excellent)
Accuracy of Scores	1 (very poor) to 5 (excellent)
Overall Satisfaction	1 (very poor) to 5 (excellent)

Participants were also asked for free-text suggestions, which were used to refine the interface and the wording of the feedback.